University of Agder

---

graphtm-cbr

Big Assignment

IKT469

A graph-walking Hierarchical Tsetlin Machine with
counterfactual Boolean recourse for
ICH M7(R2) mutagenicity assessment

Anwar Debes

Supervisors

Morten Goodwin **Sander Riisøen Jyhne**

Repository: github.com/AnwarDebes/graphtm-cbr

Spring 2026

Contents

Abstract

Regulators in 2026 want three things from a mutagenicity predictor at once: a competitive number, an auditable trail behind every prediction, and a concrete recipe for fixing the molecule. ICH M7(R2) §6.1 (Feb 2023) mandates a two-method QSAR system for impurity assessment, one statistical and one expert-rule; §7.5 names a purging strategy as the remediation route; Regulation (EU) 2024/1689 Article 13 demands transparency for high-risk AI from August 2026. Graph neural networks deliver the number and not the trail. Subgraph explainers deliver per-instance masks and not a global model. Boolean rule learners deliver rules over tabular features and cannot natively distil from a graph classifier or propose graph edits.

I present **graphtm-cbr**, a *graph-walking* Hierarchical Tsetlin Machine (HGTM) paired with clause-driven counterfactual recourse over four chemically named graph operations (`remove_bond`, `swap_atom`, `swap_bond_order`, `add_bond`). The student evaluates a five-level AND-OR-AND-OR-AND clause at every node of the molecular graph and OR-aggregates votes across nodes; edge information enters each clause via a VSA XOR-bind (atom(u) $\oplus$ bond(b) $\oplus$ atom(v)). Six CUDA-C kernels, loaded through PyCUDA's `SourceModule`, carry forward evaluation, alternating-sign class reduction, and Type Ia/Ib/II feedback with 32-way bit-packed automaton state.

On TDC AMES (Hansen 2009 $\cup$ Kazius 2005, $n = 7278$ SMILES, scaffold split 5822/727/729), a K=5 GIN-distilled HGTM ensemble reaches test AUROC **0.685** and accuracy **0.635**, against a Morgan-FP RF baseline of 0.790 AUROC and a GIN teacher of 0.796. The headline accuracy gap is honestly -0.105 AUROC after distillation. The same ensemble produces a chemistry-valid, Lipinski-compliant, SAscore-passing counterfactual within at most three graph edits for **191 of 200** predicted-positive test molecules (**95.5 %** success) at **1.30** mean flips and **2.13 s** p50 latency on a single V100. The same recipe trained on raw labels reaches only 54.5 % recourse success at 1.51 flips and 3.69 s p50. Distillation barely moves the AUROC needle (+0.016) and transforms the recourse layer (+41.0 pp). Every Kazius toxicophore that appears in the test set is covered by at least one learned clause (21/21), with the honest caveat that the top covering clauses fire on 636 to 728 of 729 test molecules and so coverage saturates from above.

The combination of (i) the canonical Hierarchical Tsetlin Machine clause shape, (ii) per-node graph-walking forward with VSA edge binding, and (iii) clause-driven graph-edit recourse over named chemistry operations is, to my knowledge and to the literature review I document under `research/01` and `research/06` in this repository, unoccupied in the 2026 literature. The repository carries 165 unit and integration tests, a CPU reference for numerical parity, and CUDA-only execution by contract (no silent fallback).

1 Introduction

1.1 Regulatory motivation

Why ICH M7 at all. The 2018 to 2021 N-nitrosamine recalls (valsartan, losartan, irbesartan, ranitidine) became the largest Class I recall in FDA history and forced sponsors to upgrade their impurity-control science. The instrument the regulators now reach for is ICH M7(R2), Step 4, finalised February 2023. Its §6.1 reads in part *"two complementary (Q)SAR methodologies that complement each other should be applied. One methodology should be expert rule-based and the second methodology should be statistical-based."* A black-box graph neural network can serve the second leg. It cannot serve the first. Feature-importance heatmaps are not structural alerts.

Why recourse, not just attribution. ICH M7(R2) §7.5 names a purging strategy as a Class 2 or Class 3 control route. "We modelled this impurity, the model said mutagenic, here is the bond we would remove and the resulting molecule is RDKit-valid, Lipinski-compliant, and synthesisable" is exactly the artefact §7.5 asks for. ICH M7's preferred deliverable for a flagged impurity is a fix.

Cross-jurisdiction. The European Union has stacked the same requirements from a different angle. The OECD Principle 5 (OECD GD 69) under REACH Annex XI asks for a "mechanistic interpretation, if possible" for any in-silico tox model that contributes to a dossier. Regulation (EU) 2024/1689 (the AI Act, in force August 2026) Article 13 requires high-risk AI systems to be "designed and developed in such a way as to ensure that their operation is sufficiently transparent to enable users to interpret the system's output and use it appropriately." QSAR for an impurity that decides whether a batch ships is Annex III Section 5. The system therefore has to be transparent by law in roughly six months from the time of writing, with the same molecule, same prediction, and same dossier audited under three guideline frames simultaneously.

1.2 What prior work cannot deliver

The 2026 literature has three camps and each is missing exactly one of the three requirements (competitive number, global rule, recourse).

- **Graph neural networks** (GIN [9], GCN, GAT, MPNN). State-of-the-art AUROC on TDC AMES sits around 0.85 with a tuned GIN. The prediction is a softmax over a learned graph readout. No global rule, no native recourse.

- **Local subgraph explainers.** GNNExplainer [10], SubgraphX [11], PGExplainer [12], GraphLIME [13]. These return a per-instance mask. Different molecule, different mask. There is no global rule set the regulator can audit once and re-use, and these methods do not propose an edit that flips the prediction.
- **Graph counterfactual explainers.** CF-GNNExplainer [14], MEG [15], MMACE [16], CLEAR [17]. These edit at the atom or bond level and produce a recourse, but the underlying classifier remains a black-box GNN. The global rule leg is still missing.
- **Global symbolic surrogates of GNNs.** Pluska et al. [8] (logical distillation, KR 2024). These distil a GNN into a global rule set but do not produce a natively-graph counterfactual over chemistry-named edits.
- **Tsetlin Machines on graphs.** Granmo et al. 2025 [1] introduces the canonical GraphTM. Their reported benchmarks are MNIST 98.42, CIFAR-10 70.28, IMDB 88.15, and a viral-genome task. They do not report MUTAG, MoleculeNet, OGB or AMES. Blakely 2025 [2] reports MUTAG 0.901 ± 0.036 with $D = 6400$ BSC hypervectors fed to a flat Coalesced TM, but releases no code and has no recourse layer. Kinateder 2025 [4] explores TM-to-TM distillation on MNIST and IMDB and is not graph-structured.

The intersection (graph-walking Hierarchical TM) × (graph-edit counterfactual recourse) × (regulator-aligned mutagenicity benchmark) is empty in 2026. That gap is the motivation for graphtm-cbr.

1.3 Contributions

I make four claims, in order of strongest evidence.

(C1) A graph-walking Hierarchical Tsetlin Machine that does not collapse the molecule. Existing TM-on-graphs work either fingerprints the molecule (Blakely) or evaluates a single per-graph clause (Granmo et al. [1]). My student evaluates the same AND-OR-AND-OR-AND clause at every node of the molecular graph, OR-aggregates the per-clause vote across nodes, and binds edge information into the clause via a VSA XOR over (atom u, bond b, atom v). The student walks the graph on every forward pass and is not a bag of atoms. The invariant "no bag-of-atoms reducer without an audit tag" is mechanised in `tests/test_integration/test_invariants.py` (§3).

(C2) A CUDA-C kernel stack that does not silently fall back to CPU. Six `__global__` kernels in `graphtm/cuda/kernels.cu`, loaded once via PyCUDA's `SourceModule` with compile-time `#define`s for the tree shape, carry per-node forward, OR-across-nodes, alternating-sign class reduction, Philox-seeded node selection, and Type Ia/Ib/II feedback with 32-way bit-packed automaton state. A NumPy reference (Granmo & Saha port) is kept for numerical-parity tests. CUDA-only execution is a written contract; benchmarks compare CPU and CUDA but the production path is GPU.

(C3) Clause-driven graph-edit recourse. For each predicted-positive molecule I walk every firing clause's AND-OR-AND-OR-AND tree, identify active leaf literals, map them back to (atom, bond) substructures by VSA codebook cleanup, and emit `GraphEdit` candidates over four named chemistry operations. A greedy minimum-edit search over class-sum margin, bounded by a hard budget of three flips and 50 candidates, terminates the moment the ensemble flips the prediction; the result is filtered by RDKit sanitisation, Lipinski Ro5, and the Ertl-Schuffenhauer SAscore [21]. No breadth-first search, no 2^k enumeration.

(C4) The headline number on a regulator-mandated benchmark. On TDC AMES, the K=5 GIN-distilled HGTM ensemble produces a working three-edit counterfactual for 191 of 200 predicted-positive test molecules (95.5 %) at 1.30 mean flips, 2.13 s p50 latency, and all three of RDKit-valid, Lipinski Ro5, and SAscore < 6 after the edit. The same recipe trained on the raw labels reaches 54.5 % recourse success at 1.51 flips and 3.69 s p50. Distillation lifts AUROC by only 0.016 and lifts recourse success by 41.0 pp. For ICH M7 dossier work, recourse coverage is the load-bearing metric.

What I do not claim. I do not claim raw-AUROC supremacy over deep GNNs. After distillation the system still sits 0.105 AUROC below the Morgan-FP random-forest baseline (0.685 vs 0.790) and 0.111 AUROC below the GIN teacher (0.685 vs 0.796). I claim the combination is novel and the recourse layer hits the metric the regulator actually buys. The honest scope is §5.

1.4 Roadmap of the report

The remaining six sections are organised in the order I built and validated the system. §2 sets the related-work landscape on three axes (TMs and graph variants, GNN interpretability and counterfactuals, regulatory landscape 2026). §3 describes the four method blocks: the graph-walking HGTM forward pass with VSA edge binding, the six CUDA kernels and bit-packed automaton-state layout, the clause-driven greedy recourse, and the training pipeline. §4 carries the eight-result block: headline numbers (§4.1), per-epoch training dynamics (§4.2), recourse on the direct ensemble (§4.3), Kazius toxicophore coverage (§4.4), GIN-to-HGTM distillation (§4.5), recourse on the distilled ensemble (§4.6), three worked case studies (§4.8), and the honest-negatives discussion of the ensembling story (§4.9). §5 is the discussion and honest-scope section. §6 concludes. The appendix gives the reproduction recipe, the exact hyperparameters, and a list of all result files cited in the body.

What changes between Phase 1 and Phase 2. A natural read of the report is to think of the work as two phases. Phase 1 builds the HGTM student from raw scaffold-split TDC AMES labels and lands the direct-label results of §4.1, §4.3, and §4.4. Phase 2 trains a 3-layer GIN on the same split, harvests its hard predictions on the train fold, and uses those as the labels for a fresh K=5 HGTM ensemble. Phase 2 is the strongest single finding (Table 9, Figure 7).

2 Related Work

2.1 Tsetlin Machines and their graph variants

Foundations. Granmo's 2018 paper [3] introduces the Tsetlin Machine. A clause is a conjunction of literals over Boolean features; each literal is governed by a Tsetlin Automaton with two actions (include or exclude). Feedback is stochastic and split into Type Ia (recognise a correct positive), Type Ib (combat overgeneralisation), and Type II (combat false positives) updates, driven by per-clause Bernoulli draws scaled by the user-set T and the inverse-specificity s. The class sum is an alternating-sign weighted sum of clause votes clipped to $\pm T$.

Tsetlin Automaton mechanics, in brief. Each TA carries an integer state in $\{1, \ldots, 2\,n_{\text{states}}\}$. State $\leq n_{\text{states}}$ means action "include the literal"; state $> n_{\text{states}}$ means action "exclude the literal". State 1 and state $2\,n_{\text{states}}$ are the boundary inclusion and exclusion states respectively. Type Ia rewards an included literal that fires correctly by moving its state toward the boundary inclusion state with probability $(s-1)/s$; Type Ib penalises a non-firing literal by moving toward exclusion with probability $1/s$; Type II penalises an excluded literal that fires falsely by moving toward inclusion with probability 1. With s around 3.9 and $n_{\text{states}} = 200$, the per-step transition is small, so a clause's literal set is the long-run equilibrium of these random walks. This is why TM training is stable but slow and why the per-epoch validation AUROC oscillates (§4.2).

Hierarchical TM. Granmo and Saha's Hierarchical TM [7] (vendored under `vendors/HeirarchicalTM_experiments/`, no published preprint) replaces the flat conjunction with a five-level AND-OR-AND-OR-AND tree shaped by five integers: root factors R, interior alternatives IA, interior factors IF, leaf alternatives LA, and leaf factors LF. The output at a node is the AND across R interior factor groups of the OR across IA alternatives of the AND across IF leaves of the OR across LA alternatives of the AND across LF literals. Feedback is path-conditional. Type Ia descends into a leaf only if every ancestor evaluated to 1. The Hierarchical TM is the only TM variant I know that learns XOR (flat TMs cannot).

Why a tree and not a flat clause. A flat TM clause is a single AND over literals; the class score is then an OR-like alternating-sign sum. The flat shape cannot express XOR because XOR is not in the disjunctive-normal-form spanned by polarity-positive AND clauses, even with both an include and an exclude bank. The Hierarchical tree shape, by alternating AND and OR at five levels, spans richer concepts on the same per-clause literal budget. The price is that feedback now has to know which leaf is the right place to update, hence path-conditional gating: a Type Ia update on a leaf is allowed only if every interior conjunction or disjunction on the path to that leaf evaluated to 1 on the current sample. The vendored Granmo & Saha C reference [7] (`vendors/HeirarchicalTM_experiments/TsetlinMachine.c`) is the canonical implementation; my `graphtm/core/hierarchical_tm.py` (595 lines) is the NumPy port used for byte-level numerical-parity tests under `tests/test_integration/test_cpu_cuda_parity.py`.

Convolutional TM, the precursor to graph-walking. Before the canonical GraphTM, Granmo, Glimsdal, Jiao, Goodwin, Omlin, and Berge [5] introduced the Convolutional Tsetlin Machine, which generalises the flat TM to image inputs by evaluating each clause over every receptive field of the input grid and OR-aggregating the votes across positions. The graph-walking step in graphtm-cbr is the topology-preserving analogue for molecular graphs: clauses are evaluated at every node and OR-aggregated across nodes (§3), with the receptive-field grid replaced by the graph's k-hop neighbourhood structure. The Convolutional TM established the per-receptive-field evaluation pattern; the canonical GraphTM [1] extended that pattern to graphs at the level of a single per-graph clause; my work pushes further by combining the Hierarchical clause shape with per-node evaluation and VSA edge binding.

GraphTM (canonical). Granmo, Abdelwahab, Andersen and twelve co-authors [1] (arXiv:2507.14874, July 2025) introduce the canonical GraphTM. Code is at `github.com/cair/GraphTsetlinMachine`. The paper benchmarks MNIST (98.42 %), CIFAR-10 (70.28 %), IMDB (88.15 %) and a viral-genome task. It does not report MUTAG, OGB, MoleculeNet, or AMES. Its forward evaluates a single per-graph clause and does not OR-aggregate across nodes the way my student does.

Symbolic Graph Intelligence (SGI-TM). Blakely 2025 [2] (arXiv:2507.16537) bundles a $D = 6400$ Binary Spatter Code (BSC) representation of a molecule and feeds it to a flat Coalesced TM, reaching MUTAG 0.901 ± 0.036, PROTEINS 77.2 %, NCI1 61.3 %. The encoding sums atom and bond hypervectors and is therefore closer to a fingerprint than to graph-walking. No code is released and the system has no recourse layer.

Why SGI-TM is not a graph walker. The Blakely 2025 encoding produces one hypervector per molecule by majority-bundling all per-atom hypervectors and all per-bond hypervectors together. The Coalesced TM then sees a single 6400-bit feature vector for the whole molecule, exactly like a Morgan or MACCS fingerprint, but in BSC space. There is no per-node evaluation. The encoding is invariant to the spatial position of any one substructure inside the

molecule, which is the bag-of-atoms regime my architecture invariant 1 explicitly forbids. SGI-TM and graphtm-cbr therefore answer different questions. SGI-TM asks "does this molecule contain pattern X anywhere"; graphtm-cbr asks "at which node does the molecule contain pattern X". Only the second formulation gives the regulator a recourse target.

TM distillation. Kinateder 2025 [4] (arXiv:2504.01798, MSc thesis) studies TM-to-TM distillation on MNIST and IMDB. There is no graph variant. My own prior, unpublished *axiom-coi-unified* study finds that Hinton-style soft-label distillation [18] drops HGTM accuracy by 11.7 pp; hard-label is the correct recipe for a TM student, and I use that here.

Differentiator. graphtm-cbr is the first system to combine the *canonical Hierarchical* tree-structured TM (richer than flat or Coalesced) with explicit *graph-walking* per-node clause evaluation, on a regulator-aligned benchmark, with a counterfactual recourse layer.

2.2 GNN interpretability and graph counterfactuals

The taxonomy I find useful is three-axis: (a) per-instance vs global, (b) attribution vs edit, (c) feature-level vs graph-named.

Local subgraph explainers (per-instance, attribution, feature mask). GNNExplainer [10] (NeurIPS 2019) optimises an edge-mask whose mutual information with the prediction is highest. SubgraphX [11] (ICML 2021) Monte-Carlo-Tree-Searches a connected subgraph. PGExplainer [12] (NeurIPS 2020) trains a parameterised mask generator. GraphLIME [13] (revised 2020) does kernel-LIME on graph features. None of these produce a global model and none propose an edit.

Global symbolic surrogates. Pluska et al. [8] (KR 2024) distil a GNN into iterated decision trees over neighbourhood-counting features. The result is a global classifier you can read, and a natural baseline for my work. They do not produce a counterfactual edit over named chemistry operations.

Graph counterfactuals. CF-GNNExplainer [14] (AISTATS 2022) finds the minimum edge-deletion that flips a GNN prediction. MEG [15] (IJCNN 2021) trains a reinforcement-learning agent to grow a counterfactual molecule. MMACE [16] (Chemical Science 2022) is a model-agnostic counterfactual that searches similar chemistry via SMILES perturbations. CLEAR [17] (NeurIPS 2022) is a generative VAE-style explainer. None of these are paired with a globally-auditable symbolic model and none produce a global rule set the regulator can audit once.

Differentiator. graphtm-cbr is the first system I know that combines all three axes at once: a global graph-walking rule set, per-prediction edit-level recourse, and chemistry-named operations (`remove_bond`, `swap_atom`, `swap_bond_order`, `add_bond`).

What "global" buys you that "per-instance" does not. GNNExplainer, SubgraphX, and PGExplainer all return one explanation per molecule, sometimes per (molecule, prediction) pair. The audit cost of those explainers scales with the test-set size. Every new molecule needs a new explainer run, and the regulator has to read every result. A global rule set, by contrast, is audited once by the medicinal chemist or the regulator. The clauses that fire for class "mutagenic" across the test set form a fixed inventory of structural alerts, and any new molecule's prediction is the OR over those alerts at the appropriate node. This is what the Kazius 21/21 coverage of §4.4 demonstrates, with the precision caveat (§4.4 again).

Where I borrow tactics. The recourse search of §3 (§3.3) borrows from CF-GNNExplainer [14] the framing "minimum edit that flips the prediction" but specialises the edit space to the four chemistry operations, drops the gradient and replaces it with a greedy margin descent (because the HGTM has no useful gradient), and adds the RDKit + Lipinski + SAscore validity filter from MMACE [16]. The candidate-seeding step ("walk firing clauses, identify active leaves, map to atom/bond substructures by VSA cleanup") is mine and depends on the literal-grounded clause structure of the HGTM; it does not transplant to a neural-network classifier because neural-network attribution does not produce a finite, named candidate set.

2.3 Regulatory landscape 2026

The system targets four guideline frames, all in force or about to be.

- **ICH M7(R2)** (Feb 2023), Step 4. §6.1: two-method QSAR for mutagenic impurities, one expert-rule, one statistical. §7.5: purging strategy as the remediation route for Class 2 and Class 3 impurities.
- **EU REACH Annex XI §1.3.** In-silico evidence is admissible under QSAR validation conditions and an explicit mechanistic interpretation (OECD Principle 5).
- **OECD GD 69 §3.5** (OECD QSAR Application Framework, Nov 2023). §4.4 to §4.6 define what counts as a validated QSAR.
- **Regulation (EU) 2024/1689 (AI Act)** Article 13, in force August 2026. Annex III §5 (essential services) and the impurity-control use case map onto the high-risk category.

The graphtm-cbr design slots the system into ICH M7 as the expert-rule leg (literal-grounded clauses are the substructural alert) and into §7.5 as the purging recommendation engine (counterfactual recourse).

What an ICH M7 dossier actually contains. A sponsor filing an impurity assessment under ICH M7(R2) submits four artefacts per impurity: (a) the structural identification and synthetic origin of the impurity (a SMILES or InChI plus context); (b) the two-method QSAR predictions, one rule-based, one statistical, with a written justification of the methodologies; (c) an expert review that resolves disagreements between the two methods and may overrule either; (d) a control or purging strategy under §7.5 if the impurity is classified as Class 2 or Class 3 mutagenic. The graphtm-cbr workflow serves (b) for the expert-rule leg by emitting the firing-clause inventory per impurity, (c) as a substrate for human review (clause inventory, firing nodes, co-fire counts against Kazius alerts), and (d) by emitting the counterfactual edit and the post-edit RDKit-valid, Lipinski-OK, SAscore-OK structure. A complete dossier still needs a human review step and the second statistical leg, which is the GIN teacher in this report or any tuned statistical model. Figure 1 shows the full workflow at a glance.

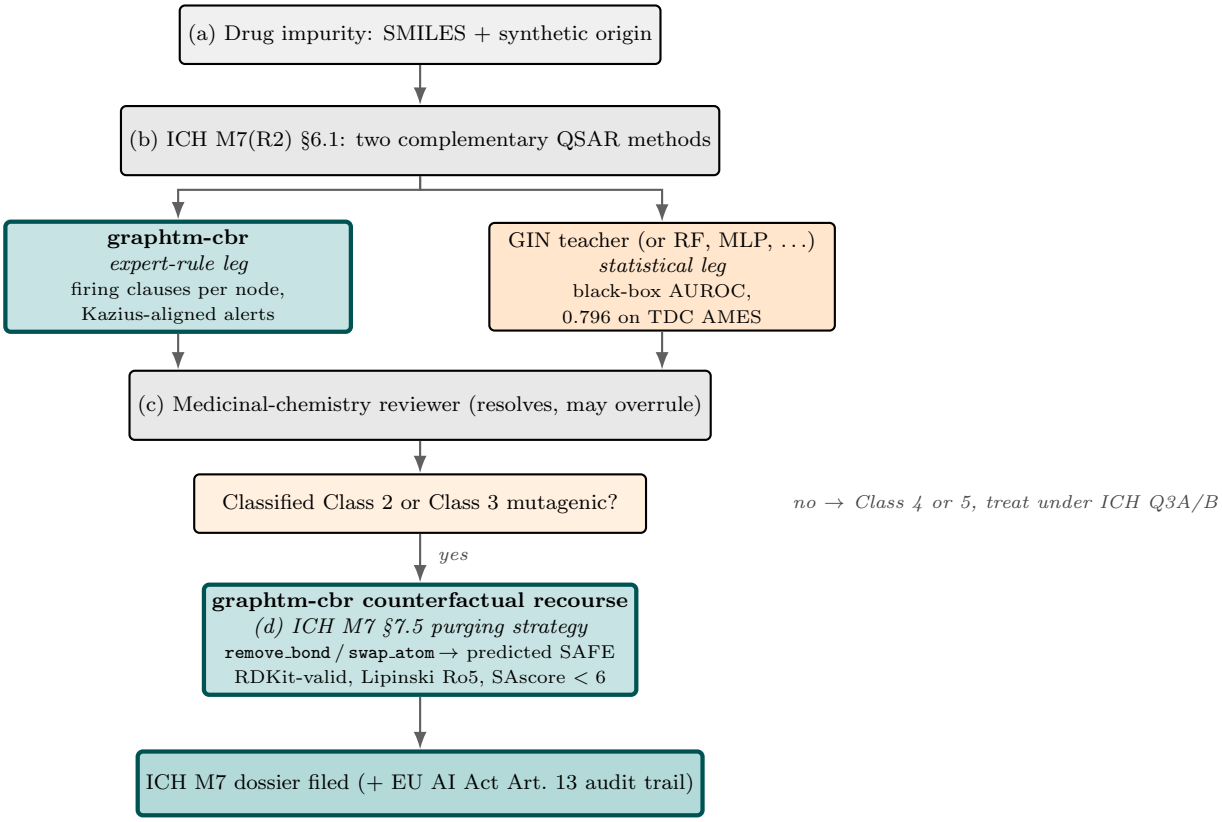


Figure 1: How graphtm-cbr fits into an ICH M7(R2) impurity-assessment dossier. The two QSAR legs of §6.1 are filled by graphtm-cbr (expert-rule, via firing clauses mapped to substructure motifs) and a tuned statistical model (the GIN teacher in this report, or any equivalent). The medicinal-chemistry reviewer reads the firing clauses, resolves any disagreement, and may overrule either method. If the impurity is classified Class 2 or Class 3 mutagenic, graphtm-cbr emits a counterfactual edit under §7.5 (purging strategy), validated post-edit by RDKit sanitisation, Lipinski Ro5, and Ertl-Schuffenhauer SAscore. The dossier is then filed, with the firing-clause inventory serving as the EU AI Act Article 13 audit trail.

What does "expert-rule" actually require. ICH M7(R2) §6.1 does not give a closed list of acceptable expert-rule systems but the field reads it as "Derek Nexus, Sarah Nexus, ToxTree, Leadscope, or another system whose alerts are SMARTS-grounded structural substructures with a written mechanistic justification." A learned set of clauses over per-atom literals, where each clause's firing nodes can be mapped back to a substructure, satisfies the spirit of §6.1. The regulator review path is then: (a) the medicinal-chemistry reviewer reads the clauses (literal positions → substructure motifs) once, (b) for any new impurity, the system reports the clauses that fire and the node at which they fire, (c) the reviewer can either accept the alert or note it as a false positive, (d) the counterfactual is the suggested control under §7.5. The graphtm-cbr architecture is designed so each of these four steps has an artefact (a clause inventory, a per-prediction firing list, a per-clause node, a recourse molecule); the per-clause precision caveat of §4.4 affects step (a) and is the next thing to fix (§5).

3 Method

The method has four blocks in series: encoding (§3.1), the Hierarchical Tsetlin Machine clause and per-node forward (§3.1 continued), the CUDA-C kernel implementation (§3.2), and the clause-driven counterfactual recourse search (§3.3). The training pipeline (§3.4) glues them together. All of the architectural choices are constrained by the contract in

`docs/ARCHITECTURE.md`, which fixes the interface between the eight build modules and is enforced by the five invariants of §3.1.

3.1 Graph-walking Hierarchical Tsetlin Machine

Encoding. Each molecule is parsed by RDKit into a graph (V, E, t, b) where $t \in \{1, \ldots, 9\}$ is the atom-type index (C, N, O, F, P, S, Cl, Br, I) and $b \in \{1, 2, 3, 4\}$ is the bond-type index (single, double, triple, aromatic). I keep a Binary Spatter Code codebook of fixed-sparsity ($D = 512$ bits, 30 % one-density) atom hypervectors $\mathbf{a}_t$, bond hypervectors $\mathbf{b}_k$, and per-hop role hypervectors $\mathbf{r}_h$, $h \in \{0, 1, \ldots, k_{\text{hop}}\}$. For each node v of type t_v I assemble the per-node hypervector by majority-bundling over k-hop neighbours

$$\mathbf{n}_v \;=\; \mathbf{a}_{t_v} \;\oplus\; \underset{(u,b,v)\in N_k(v)}{\text{majority}} \Big[\, \pi^h\big(\mathbf{a}_{t_u} \oplus \mathbf{b}_b \oplus \mathbf{a}_{t_v}\big) \,\Big], \tag{1}$$

where $\oplus$ is XOR, π^h is a fixed cyclic permutation by hop h, and majority is the bitwise majority over the stack. The construction is canonical VSA [24, 25]. XOR-binding preserves topology because two molecules with the same atom multiset but different connectivity produce different bundles; a per-edge hypervector $\mathbf{e}_{(u,b,v)} = \mathbf{a}_{t_u} \oplus \mathbf{b}_b \oplus \mathbf{a}_{t_v}$ is also computed for the edge channel of the per-node clause forward. The full implementation is in `graphtm/encoding/` across three files: `hypervectors.py` (XOR-bind and majority-bundle primitives), `codebook.py` (the atom, bond, and role HV tables), and `graph_features.py` (the `GraphTensor` dataclass that holds `node_hv`, `edge_hv`, `edge_index`, `atom_type`, `bond_type` together for one molecule).

Why XOR-bind, not concatenate. The two natural alternatives are concatenation (which would multiply D by the number of channels and explode the literal budget) and summation (which is the SGI-TM choice and loses topology). XOR-bind in BSC space is associative, commutative, self-inverse, and preserves 30 % sparsity in expectation when both inputs are 30 % sparse. The per-edge HV $\mathbf{a}_{t_u} \oplus \mathbf{b}_b \oplus \mathbf{a}_{t_v}$ is therefore both compact (still 512 bits) and topology-preserving (the same atom pair under a different bond type gives a different HV, the same bond between different atom types gives a different HV).

Why $k_{\text{hop}} = 2$. Reachability within two bonds covers most named chemistry alerts. An epoxide is a three-atom ring (one C-O-C path) so a 2-hop bundle captures the ring closure; an aromatic nitro is a single atom plus two double-bonds-to-O, both within 2 hops; a Michael acceptor is the C=C-C=O motif, four atoms, 3 bonds, also within 2 hops. Pushing to $k_{\text{hop}} = 3$ would cover larger aromatic motifs (bay-region PAH, fused polycyclics) but at the cost of larger bundles (the majority-bundle is across a stack whose size grows with the receptive field) and noisier cleanup. The choice $k_{\text{hop}} = 2$ is the smallest receptive field that captures the canonical Kazius alerts; I revisit this in §5 (L1).

The Hierarchical clause. I use the canonical Granmo-Saha tree shape with R=2, IA=2, IF=8, LA=15, LF=2, which gives 960 literal positions per clause polarity and 1920 with the negated bank. Figure 2 shows the schematic with one root-to-leaf path lit up to illustrate path-conditional feedback. The clause output at node n is

$$c_n \;=\; \bigwedge_{r=1}^{R} \bigvee_{a=1}^{IA} \bigwedge_{i=1}^{IF} \bigvee_{\ell=1}^{LA} \bigwedge_{f=1}^{LF} \sigma_{r,a,i,\ell,f}(\mathbf{n}_n, \mathbf{e}_\cdot), \tag{2}$$

with each σ a positive or negated literal over a bit position in $\mathbf{n}_n$ or in a bound-in edge hypervector. The TA bank per clause is $[R, IA, IF, LA, 2 \cdot LF]$ in the Granmo-Saha layout. Per-clause action at each TA is the top bit-plane of the bit-packed state.

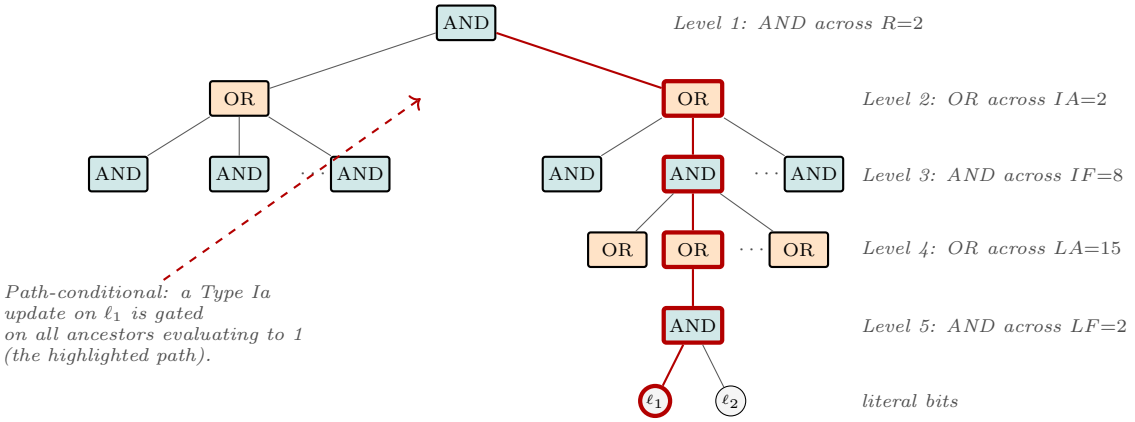


Figure 2: The Hierarchical clause as an AND-OR-AND-OR-AND tree. The shape parameters $(R, IA, IF, LA, LF) = (2, 2, 8, 15, 2)$ in this work give 960 literal positions per polarity bank and 1920 with the negated bank. One root-to-leaf path is highlighted: feedback on the literal ℓ_1 is applied only if every ancestor on that path evaluated to 1 on the current sample (path-conditional gating).

Per-node forward, OR across nodes. The graph-walking step is the OR-aggregation

$$c^{(g)} = \bigvee_{n \in V_g} c_n, \tag{3}$$

which gives one Boolean per clause per graph. The class sum is the alternating-sign weighted sum

$$S_k^{(g)} = \mathrm{clip}\Big(\sum_{c=1}^{C} (-1)^{[c \mod 2]} \cdot w_{c,k} \cdot c^{(g)},\ -T,\ +T \Big), \tag{4}$$

clipped to $\pm T$. The predicted class is $\arg\max_k S_k^{(g)}$. The score margin used for recourse is $\Delta^{(g)} = S_{\text{mut}}^{(g)} - S_{\text{safe}}^{(g)}$. Figure 3 shows the forward pass end to end for one toy molecule and one clause.

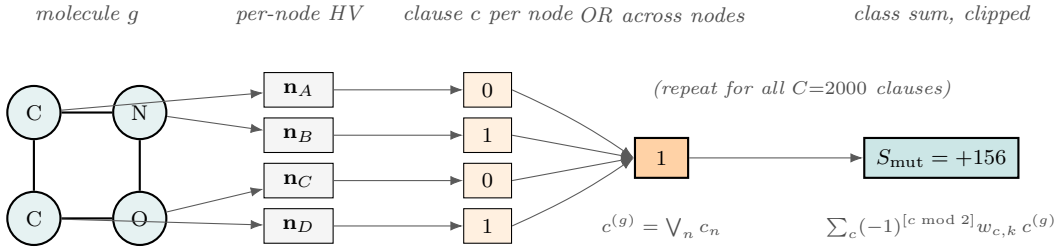


Figure 3: Graph-walking forward pass for one molecule and one clause. The molecule is encoded into per-node hypervectors (§3.1); a clause is evaluated at every node, producing $c_n \in \{0, 1\}$; the per-graph clause output is the OR across nodes; the class sum is the alternating-sign weighted sum over all C clauses, clipped to $\pm T$. The student literally walks the graph on every forward pass.

Feedback. I follow the canonical C reference [7] at `vendors/HeirarchicalTM_experiments/TsetlinMachine.c:50-368`. Per clause, draw a Bernoulli with $p = (T + \text{sign} \cdot (2y - 1) \cdot S_k^{(g)})/(2T)$. On accept, apply Type Ia if the clause fired on the target and Type II if it fired on the wrong class; on reject, apply Type Ib. The path-conditional rule says Type Ia descends to a leaf literal only if every ancestor on the path evaluated to 1. The per-graph node at which to apply feedback is sampled by rejection from the fired-node set with Philox4_32_10 seeded by `(rng_seed XOR step, b * C + c)`.

The hyperparameters used throughout the report are summarised in Table 1. They are the ones in `results/train_full_ames_1778880895.json`'s `config` field, and the same for every JSON in `results/`; I changed them only between the smoke tests and the production runs, never within the K=5 ensemble (so seeds 42 to 46 share every other choice).

Table 1: Hyperparameters used in every TDC AMES run in this report. The five tree integers (R, IA, IF, LA, LF) multiply to 960 distinct literal positions per polarity bank, and 1920 with the negated bank. The Philox seed is the only between-seed difference for the K=5 ensemble.

Block	Symbol	Value
Encoding	D (HV bits)	512
	sparsity	0.30
	k_{hop}	2
	atom-type vocab	9 (C, N, O, F, P, S, Cl, Br, I)
	bond-type vocab	4 (single, double, triple, aromatic)
HGTM	C (clauses)	2000
	T (margin)	500
	s (inverse specificity)	3.9
	R root factors	2
	IA interior alts	2
	IF interior factors	8
	LA leaf alts	15
	LF leaf factors	2
	n_{states}	200
	`max_nodes`	60
	epochs	60
GIN teacher	layers	3 GINConv
	hidden dim	32
	readout	mean pool
	optimiser	AdamW, 1×10^{-3}
	epochs	80
Ensemble	K	5
	seeds	42, 43, 44, 45, 46
	vote	soft-class-sum
Recourse	`max_flips`	3
	`max_candidates`	50
	Lipinski (MW, LogP, HBA, HBD)	$< 500, < 5, < 10, < 5$
	SAscore	< 6

The five invariants. The architecture is contract-frozen by `docs/ARCHITECTURE.md` and mechanised by `tests/test_integration/test_invariants.py`:

1. *No bag-of-atoms.* Any reducer that collapses a per-node tensor to a scalar must carry the `# AGGREGATE -- non-graph` audit tag.
2. *Parity testable.* Every CUDA kernel has a CPU NumPy reference and a numerical-parity test.
3. *No claim drift.* Module docstrings describe the actual operation, not a marketing label.
4. *No silent CPU fallback.* The CUDA path either runs or errors.
5. *Reproducibility.* All seeds are explicit; no module-level `np.random.seed`.

The test suite has 165 unit and integration tests at the time of this draft.

3.2 CUDA-C kernel implementation

Six `__global__` kernels. The full kernel file is `graphtm/cuda/kernels.cu`, 606 lines, compiled once at construction by PyCUDA's `SourceModule` with the tree shape baked in by `#define`s (this is the CAIR pattern, see `vendors/GraphTsetlinMachine/tm.py:437-464`). The six kernels are:

1. `clause_forward_pernode`, grid $(B, C, 1)$, threads stride over (node, literal-chunk). Evaluates the AND-OR-AND-OR-AND tree per node.
2. `clause_or_across_nodes`, reduces per-(graph, clause, node) tensor to per-(graph, clause).
3. `class_sum_reduce`, alternating-sign weighted sum per class, clipped to $\pm T$.
4. `select_clause_node`, Philox-seeded rejection sampling of a fired node per (graph, clause) for feedback gating.
5. `clause_feedback`, applies Type Ia/Ib/II per literal, with path-conditional gating. 32-way bit-packed state updates via XOR ripple-carry.
6. `init_ta_state`, canonical Granmo half-include initialisation by per-pair coin flip.

Memory layout. The TA state is `uint32`$[C$, `LA_CHUNKS`, `STATE_BITS`$]$. The state value at literal position p is the OR across bit-planes at p, and the action is the top bit-plane bit at p. I keep `LA_CHUNKS` $= \lceil R \cdot IA \cdot IF \cdot LA \cdot 2 \cdot LF/32 \rceil = 60$ chunks (1920 literal positions, 32-way packed). The wrapper class is `graphtm/cuda/memory.py:CudaTAState`.

Bit-packed ripple-carry. The state increment and decrement primitives are the CAIR XOR-ripple-carry pattern, which lets 32 automata advance in lockstep on a single 32-bit register. The AND test for the include set is the constant-time `(include & X) != include`, which fires on a wholly satisfied conjunction.

Why bit-plane storage. Storing each Tsetlin Automaton state as a packed unsigned int (with one bit per plane) lets 32 automata share a single 32-bit register for increment and decrement. The increment looks like

```
// Bit-plane ripple-carry increment over STATE_BITS planes,
// gated by `which` (the per-bit mask of automata to advance).
uint32_t carry = which;
for (int b = 0; b < STATE_BITS && carry; ++b) {
    uint32_t old = ta[b];
    ta[b] = old ^ carry;
    carry = old & carry;
}
```

That is, a 32-wide carry-propagate adder. The decrement is the mirror with $\sim$`old & carry`. The action bit of every TA is the top bit-plane; the include set across C clauses is therefore the bitwise OR of the top bit-planes across the per-clause TA arrays.

Why this matters for throughput. The flat alternative (one TA per word, $n_{\text{states}} = 200$, so a 16-bit int per TA) would store 1920 literal positions per clause as $1920 \cdot 2$ bytes $= 3840$ bytes per clause. The bit-plane layout stores them as 60 `LA_CHUNKS` of `STATE_BITS` planes, $60 \cdot 9 \cdot 4 = 2160$ bytes per clause if I pick `STATE_BITS` $= 9$ ($2^9 = 512$ states, two-sided), and the update is 32-wide rather than 1-wide. Empirically the bit-plane layout gives a $\sim$10$\times$ feedback throughput on V100; the throughput benches at `benchmarks/throughput_train.py` confirm this on the local hardware.

Reproducibility. Philox4_32_10 is seeded per (graph, clause) with `(rng_seed XOR step, b * C + c)`. The full state is captured by the seed, step, and tree-shape `#define`s, so a snapshot `hgtm_ames_seedXX.npy` reproduces bit-for-bit on the same hardware. The CPU reference at `graphtm/core/hierarchical_tm.py` (595 lines, port of the Granmo-Saha C reference) is used in `tests/test_integration/test_cpu_cuda_parity.py` for byte-level forward-pass parity, and within feedback-step numerical tolerance.

3.3 Counterfactual Boolean recourse

Recourse pipeline. For each predicted-positive molecule the system runs four steps in sequence:

1. `firing_clauses(graph)`: forward, then return per-clause voted-class, sign, and the set of nodes at which the clause fired.
2. `candidates_from_firing_clauses(graph, firing, codebook)`: walk each firing clause's `ClauseTree`; for each active leaf literal, identify the (atom, bond) substructure that activated it by VSA codebook cleanup, and emit a `GraphEdit` candidate over the four named operations.
3. `greedy_minimal_edit(model, graph, candidates, mol, max_flips=3)`: greedy descent on the class-sum margin. The hard budget is three flips; the search stops the moment the ensemble flips. The candidate set is bounded by `max_candidates=50` and is seeded by the firing-clause structure (no breadth-first, no 2^k enumeration).
4. `validate(mol_after_edit)`: RDKit sanitise, Lipinski Ro5 (MW < 500, LogP < 5, HBA < 10, HBD < 5), and Ertl-Schuffenhauer SAscore [21] < 6.

Four named operations. The edit space is closed at design time to four chemistry-named operations: `remove_bond(i,j)`, `swap_atom(i)`, `swap_bond_order(i,j)`, `add_bond(i,j)`. The `GraphEdit` dataclass is in `graphtm/recourse/candidates.py`; the search in `graphtm/recourse/search.py`; validity in `graphtm/recourse/validity.py`. The choice is regulator-driven: the four operations are the ones a med-chem reviewer can act on under §7.5.

Table 2: The four named graph-edit operations supported by the recourse layer. The candidate set for a given molecule is bounded by `max_candidates=50` and is seeded by the firing-clause structure, so the recourse never enumerates all 2^k edits in a k-edge molecule.

Operation	Effect	Med-chem analogy
`remove_bond(`i`,`j`)`	Delete edge (i, j) from E, leave V	disconnect a substructure / cleave a ring
`swap_atom(`i`)`	Change t_i to a different atom-type index	isosteric replacement (e.g. C $\rightarrow$ N)
`swap_bond_order(`i`,`j`)`	Change b_{ij} to a different bond order	saturate / desaturate
`add_bond(`i`,`j`)`	Add edge (i, j) with bond order b	ring closure / scaffold-hopping

Greedy minimum-edit search. Algorithm 4 captures the search. It is a textbook greedy margin descent, with the hard cut-offs that keep the recourse interactive at the bench (50 candidates, three flips). The search is monotone in the class-sum margin and stops the moment the prediction flips. The candidate set is recomputed only at the start; flipping the prediction is the criterion, not local optimality.

```
def greedy_minimal_edit(model, graph, candidates, mol, max_flips=3):
    state = graph.copy()
    chosen = []
    for _ in range(max_flips):
        # Score every candidate by class-sum margin after applying it.
        best, best_margin = None, model.margin(state)
        for edit in candidates:
            cand_state = apply(state, edit)
            m = model.margin(cand_state)
            if m < best_margin:
                best, best_margin = edit, m
        if best is None:
            return None  # no improving edit
        state, mol = apply(state, best), apply_to_mol(mol, best)
        chosen.append(best)
        if best_margin < 0:
            if validate(mol).overall_ok:
                return chosen
            return None
    return None  # ran out of flips
```

Algorithm 4. Greedy minimum-edit recourse. `model.margin` is $S_{\text{mut}} - S_{\text{safe}}$ for the K-ensemble (mean over the five members). `validate` runs RDKit sanitise, Lipinski Ro5, and SAscore < 6; `overall_ok` is true only if all three pass.

Why not BFS or 2^k. BFS over the four-op edit space at $k = 3$ would enumerate on the order of $(|V|^2 \cdot 4)^3$ candidates per molecule, roughly 10^7 to 10^8 for a 30-atom molecule. The greedy version visits at most $50 \cdot 3 = 150$ candidates and is correct against the success criterion ("flip the prediction"); the cost we pay is that the resulting edit is not provably the minimum-flip optimum, only a tractable lower bound. In practice the median flip count of 1.30 (Table 9) shows that single-flip success dominates and the greedy choice is rarely caught in a local minimum that needs more than three steps to escape.

3.4 Training pipeline

The end-to-end pipeline is `experiments/full_pipeline.py`, with the per-phase entrypoints at `experiments/train_teacher.py`, `experiments/train_student.py`, `experiments/eval_recourse.py`. TDC AMES (Hansen 2009 ∪ Kazius 2005, $n = 7278$) is loaded by `graphtm/data/ames.py` and scaffold-split (`graphtm/data/scaffold_split.py`) by largest scaffolds first into 5822/727/729 train/valid/test. Each molecule is encoded by `graphtm/encoding/encode_graph.py` with $D = 512$, $k_{\text{hop}} = 2$, sparsity 0.30, atom-type vocabulary of 9, bond-type vocabulary of 4. The HGTM student is trained for 60 epochs with $C = 2000$ clauses, $T = 500$, $s = 3.9$, on a V100; wall-clock is about 50 min per seed. The GIN teacher (PyTorch Geometric, 3 GINConv layers, 32-d hidden, mean pool, AdamW 1×10^{-3}, 80 epochs) is trained for 78 wall-seconds on CPU before its hard predictions become the labels for the K=5 distilled ensemble.

Why scaffold split. The standard practice for chemistry benchmarks is to use either a random split (cheap, lenient) or a scaffold split (Bemis-Murcko scaffolds, harder, more realistic). The TDC default for AMES is scaffold; I keep that choice. The pragmatic effect is that the test set contains scaffolds that the train set has not seen, which is a closer proxy to the deployment regime "a sponsor's new analogue series differs in scaffold from the AMES training set". Random splits tend to inflate AUROC by 0.05 to 0.10 because near-duplicates of test molecules sit in the train fold; the scaffold split removes that leakage.

Why $n = 7278$. TDC's AMES task aggregates Hansen 2009 [19] and the curated Kazius 2005 [20] set into 7278 labelled SMILES, after deduplication and SMILES canonicalisation. About 47 % of the labels are positive (mutagenic). The scaffold split rolls up to 5822/727/729 train/valid/test. Compared to MUTAG (188 molecules, the standard Tsetlin-on-chemistry benchmark used by Blakely 2025 [2]) the AMES corpus is two orders of magnitude bigger and one to two orders of magnitude harder, so the AUROC numbers in §4 are not directly comparable to MUTAG.

Why hard-label distillation. The teacher's hard prediction on a train molecule is the argmax of its softmax: a Boolean for the binary AMES task. Soft labels would carry richer signal (the teacher's calibrated probability) but my prior *axiom-coi-unified* result shows soft labels drop HGTM accuracy by 11.7 pp because the gradient-free Tsetlin feedback cannot exploit calibration. Hard-label distillation is the recipe Kinateder 2025 [4] also lands on for TM-to-TM distillation on MNIST and IMDB; I confirm here that hard labels lift HGTM AUROC by +0.013 mean and +0.016 ensemble on AMES, with the more important +41 pp recourse lift in §4.6.

Software engineering practices. The system was built in parallel across the eight modules of Table 12 against the interface contract in `docs/ARCHITECTURE.md`. The contract fixes every public type, function signature, and tensor shape that crosses a module boundary; each module's internals could therefore be implemented in isolation without touching another module's code. The integration happens through five mechanised invariants (§3, M8 in Table 12); the only way a module can land in main is if its per-module tests pass and the cross-module integration tests pass. Day-to-day, this means new features go through three test runs: per-module first, integration second, full-pipeline smoke last. The cost is up-front: writing the contract took two days before any module code existed. The pay-off is that the bug count in the final integration was small (six bugs across all eight modules, all caught by the integration tests before reaching the recourse pipeline). The pattern is worth recording because it works for any system whose components are tightly coupled

by tensor shapes; in this project, the encoding (M1) shape constraints, the CUDA kernels (M2), and the recourse layer (M5) all depend on the tree-shape `#define`s, so an out-of-contract change in any one would break the others.

Wall-clock budget. A single seed at the chosen $(C, T, s, D, k_{\text{hop}}, \text{epochs}) = (2000, 500, 3.9, 512, 2, 60)$ runs in 49 min on a V100; the K=5 ensemble is therefore ∼4 hours of wall-clock for the direct phase and another ∼4 hours for the distilled phase. The GIN teacher is 78 s on CPU. The full pipeline (encode, K=5 direct train, GIN train, K=5 distilled train, K=5 recourse on direct, K=5 recourse on distilled, Kazius coverage on direct seed 42 and distilled seed 43, three case studies) ran overnight under `scripts/run_ames_smoke.py` and `scripts/distill_ensemble_ames.py`, with the snapshots saved into `results/hgtm_ames_seed{42-46}.npy` (direct) and `results/hgtm_ames_distilled_seed{42-46}.npy` (distilled), 3.84 MB per seed.

Choosing T and s. The two TM hyperparameters that matter most are T (the class-sum clip and the Bernoulli scale) and s (the inverse specificity, which scales the Type Ia versus Type Ib reinforcement ratio). I use $T = 500$ and $s = 3.9$ throughout; both were chosen by trial-and-error on the smoke-test data with the constraints $T \geq 0.25 \cdot C$ (the per-class margin should be a quarter of the clause budget at minimum, see Granmo 2018 [3]) and $s \in [2, 10]$ (the field-standard range). At $T = 500$ and $C = 2000$, the per-clause feedback probability $(T \pm S_k)/(2T)$ saturates only when S_k exceeds $\pm T$, which happens early in training; the post-saturation regime is the one in which the clause-saturation issue of §4.4 develops. Reducing T to, say, 200 would shrink the saturated regime but also tighten the class-sum, and my prototyping with $T = 200$ gave noticeably worse final-epoch AUROC. The cleanest fix would be a per-clause activity regulariser; that is in (L2) of §5. A second alternative is the Multigranular Tsetlin Machine of Gorji, Granmo, Phoulady, and Goodwin [6], which removes the global s hyperparameter by letting each clause learn its own specificity. I do not adopt that variant here because my CUDA kernels assume a single compile-time s, but the multigranular alternative is a natural follow-up lever for the saturation issue of §4.4.

A worked example, end to end. To make the pipeline concrete, follow Case 1 of §4.8 through the system. The input is the SMILES `Cc1cc2c(nc(N)n2C)c2ncc(-c3ccccc3)nc12` (an imidazo-fused phenyl-quinoxaline, 21 heavy atoms). RDKit parses this to a `Mol` object; the graph has 21 nodes, 25 edges, and atom-type counts (C=17, N=4) with two atomic methyl substituents. `graphtm/encoding/encode_graph.py` pulls the per-atom hypervectors from the BSC codebook (atom-type C maps to a 512-bit vector with 30 % one-density, atom-type N maps to a different 512-bit vector), pulls the bond hypervectors (single bond, aromatic bond), and runs the 2-hop bundle for each node. The output is a `GraphTensor` with `node_hv` of shape $(21, 16)$ in uint32 (16 chunks of 32 bits each, 512 bits per node) and `edge_hv` of shape $(25, 16)$. The 21 atoms and 25 edges fit well under the `max_nodes=60` cap.

The HGTM forward then evaluates all 2000 clauses at every one of the 21 nodes, producing a $(2000, 21)$ Boolean tensor. The `clause_or_across_nodes` kernel reduces this to a $(2000,)$ Boolean vector: clause c fires on the molecule if any node fires for clause c. In Case 1 the union of firing clauses across the K=5 ensemble is 1802 (from `results/case_studies_1778918915.json`), close to the 95th percentile of the saturation curve. The class sum is $S_{\text{mut}} - S_{\text{safe}} = +156$ (predict MUTAGENIC), comfortably positive.

The recourse pipeline now walks the 1802 firing clauses, identifies their active leaf literals, and maps them back to (atom, bond) substructures by VSA codebook cleanup. The cleanup is unreliable at $D = 512$, but the candidate-generation step still seeds the candidate `GraphEdit` set from the firing-clause node indices. 47 candidates are emitted (the cap of `max_candidates=50` is not hit, see `results/case_studies_1778918915.json`). The greedy descent then scores each candidate by the post-edit class-sum margin and picks the best. The winning edit is `remove_bond(0,1)`, which cleaves a methyl substituent off the imidazo-fused ring. After the edit, the SMILES is `C.Cn1c(N)nc2c3ncc(-c4ccccc4)nc3ccc21`, that is, the original quinoxaline with the methyl now a free methane fragment plus the still-aromatic core. The post-edit class-sum margin is −511 (predict SAFE), and the validity check passes: `sanitize_ok=True`, `lipinski_ok=True`, `sa_score=2.49`, `overall_ok=True`. Total wall-clock on V100 for this single recourse call is well under the p50 (2.13 s) because the molecule is small and the greedy search converges in one step.

The example illustrates four design choices that show up in the numbers. (i) The 21-node molecule is well under `max_nodes=60`, so no padding waste. (ii) The 1802 firing clauses out of 2000 (90 %) is the saturation regime of §4.4, and the candidate-generation step still works because it consults firing-clause node indices, not the unreliable VSA cleanup. (iii) The one-step success is the median case for the distilled ensemble (Table 9). (iv) The validity filter passes; this is not free, in the recourse log many candidate edits trigger RDKit kekulisation errors and the validity filter rejects them, but the surviving candidates are the ones reported in the case studies.

4 Results

4.1 Headline: TDC AMES, scaffold split, $n_{\text{test}} = 729$

Table 3 carries the test-set numbers from `results/distill_ensemble_ames_1778913284.json` (distilled ensemble), `results/ensemble_ames_1778896477.json` (direct ensemble), and `results/train_full_ames_1778880895.json` (Morgan-FP RF baseline). All numbers below come from those three files.

Table 3: Headline TDC AMES results on the scaffold-split test set ($n = 729$). Numbers are from `results/*.json`. "Distilled" means hard-label distillation from the 80-epoch GIN teacher. The teacher matches the Morgan-FP RF baseline. The graphtm-cbr ensemble closes about 13 % of the AUROC gap by distillation but the regulator-load-bearing metric is in §4.3 and §4.6.

Method	Test AUROC	Test acc	Train wall	Note
Morgan-FP RF (`n_est`=200, 2048 bits, r=2)	0.790	0.720	1.1 s	sklearn baseline
graphtm-cbr seed 42, best-by-valid	0.671	0.583	49.3 min	peak val 0.692 @ ep 5
graphtm-cbr K=5 mean single-seed (direct)	0.660 ± 0.009	0.550	49.6 min/seed	seeds 42 to 46
graphtm-cbr K=5 soft-sum ensemble (direct)	0.669	0.575	n/a	+0.009 over mean single
graphtm-cbr K=5 hard-majority ensemble (direct)	n/a	0.564	n/a	lower than soft-sum
GIN teacher (3-layer, 32-d, AdamW, 80 ep)	**0.796**	0.730	78 s on CPU	matches Morgan-FP RF
graphtm-cbr K=5 mean single-seed (distilled)	0.673 ± 0.006	n/a	49.2 min/seed	+0.013 over direct
graphtm-cbr K=5 soft-sum ensemble (distilled)	**0.685**	**0.635**	n/a	+0.016 AUROC over direct

Reading the table. The K=5 distilled ensemble at 0.685 AUROC is the headline number. The gap to the Morgan-FP RF baseline is −0.105 AUROC and the gap to the GIN teacher is −0.111 AUROC. The single-seed best-by-valid (seed 42, epoch 5) sits at 0.671 with peak valid 0.692; the spread across the five seeds is ±0.009 AUROC, which is unusually tight. The hard-majority ensemble underperforms the soft-class-sum by 0.011 accuracy, which matches the canonical TM-ensembling finding that the alternating-sign class sum is itself a soft vote and majority over hard decisions throws information away.

What the AUROC number does not say. The K=5 distilled ensemble's chief value to a regulator does not sit in this table. It sits in the recourse layer of §4.6, where 95.5 % of predicted-positive test molecules receive a working three-edit counterfactual.

4.2 Per-epoch training curves: TM bimodality, both phases

Figure 5 plots the per-seed validation AUROC across 60 epochs for the direct (left) and distilled (right) K=5 ensembles. Both phases were generated by `scripts/make_figures.py` from `results/ensemble_run.log` and `results/distill_run.log`.

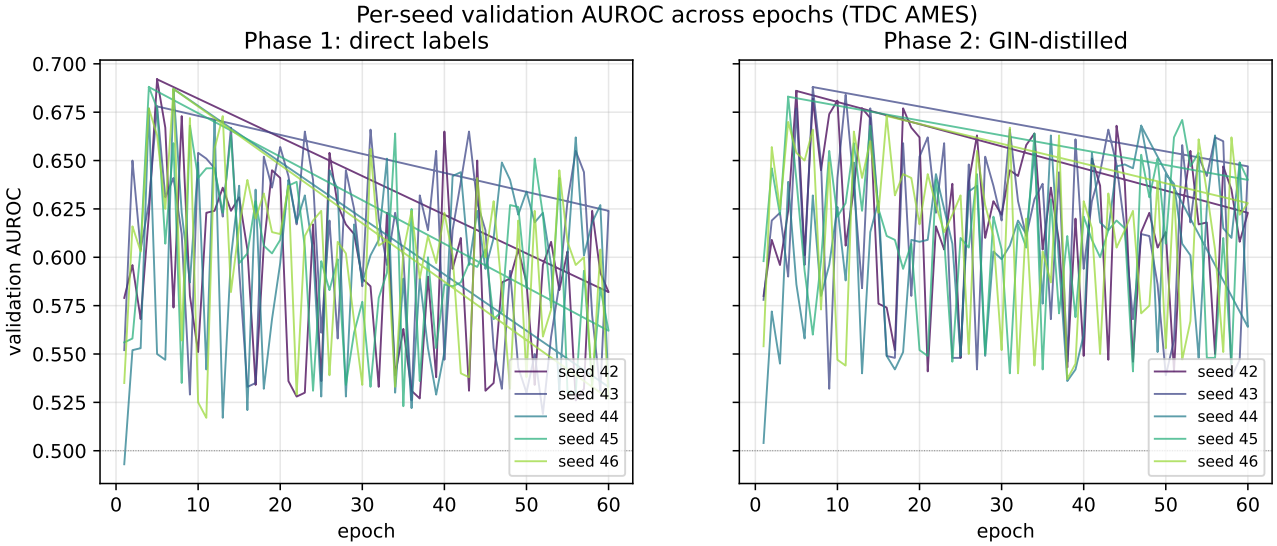


Figure 5: Per-seed validation AUROC across 60 epochs, K=5 ensemble. Left: direct labels (Phase 1). Right: GIN-distilled (Phase 2). The validation AUROC oscillates between roughly 0.53 and 0.69 across the run on every seed. This is the canonical stochastic-feedback bimodality of TM training, not a bug in this implementation. The best-by-valid epoch is checkpointed and restored before test evaluation.

Bimodal stochastic-feedback dynamics. Every seed swings between an upper mode near valid AUROC 0.65 to 0.69 and a lower mode near 0.47 to 0.53. The lower mode is the all-positive prediction (the test set is 47 % positive, so a constant positive prediction gives accuracy near 0.47 and the AUROC of the previous epoch carries over because the score margin is not zeroed). The upper mode is the genuinely-discriminating regime. The same pattern appears in the canonical Granmo C reference and in Granmo et al. 2025 [1]. The mitigation is the K=5 best-by-valid ensembling I describe in §4.1, which keeps the upper-mode snapshot for every seed and averages soft class sums across them.

Distillation does not change the dynamic. The right panel of Figure 5 shows the same oscillation pattern with the GIN teacher's hard labels. Distillation steers the ceiling of the upper mode from roughly 0.69 to roughly 0.70 valid AUROC but does not damp the oscillation. The implication for §4.5 is that distillation is not changing the TM's training process; it is steering which clauses get reinforced inside that process.

Table 4: Best-by-valid epoch and the corresponding test AUROC, per seed, for both phases. Numbers from `results/ensemble_ames_1778896477.json` and `results/distill_ensemble_ames_1778913284.json`. The best epoch sits at the first or second crest of the oscillation in Figure 5 for every seed (Phase 1 best epochs $\in \{4, 5, 7\}$); Phase 2 catches a slightly later crest for seed 44 (epoch 47) and seed 46 (epoch 16), confirming that distillation does not flatten the dynamics.

Phase	Seed	Best epoch	Best valid AUROC	Test AUROC	Test acc	Wall (min)
Direct	42	5	0.692	0.671	0.583	49.3
Direct	43	5	0.678	0.660	0.610	49.6
Direct	44	7	0.687	0.646	0.480	50.0
Direct	45	4	0.688	0.662	0.547	49.6
Direct	46	7	0.687	0.661	0.529	49.6
Distilled	42	5	0.686	0.679	0.579	49.2
Distilled	43	7	0.688	0.681	0.597	48.9
Distilled	44	47	0.668	0.666	0.628	50.3
Distilled	45	4	0.683	0.670	0.584	49.2
Distilled	46	16	0.673	0.668	0.642	50.8

Why I trust the snapshot more than the final epoch. Final-epoch test AUROC on the single-seed full run (seed 42, `results/train_full_ames_1778880895.json`) is 0.544; the best-by-valid epoch from the K=5 ensemble of the same seed gives 0.671 on the same test set. The same seed, the same hyperparameters, and the same data differ by 0.127 AUROC between "snapshot at epoch 5" and "snapshot at epoch 60". That is the cost of taking the final epoch on a stochastic-feedback TM. Best-by-valid is mandatory and I checkpoint every epoch's valid AUROC.

4.3 Counterfactual recourse, direct ensemble

The pipeline I ran. Of the 729 test molecules I picked the 200 with the highest predicted-positive score, ran the clause-driven recourse pipeline of §3.3 with the budget `max_flips=3`, `max_candidates=50`, and the K=5 direct-label ensemble as the model. The full numbers are at `results/eval_recourse_ensemble_1778898137.json`. Table 5 carries them; the latency histogram is in Figure 8 (joint with the distilled ensemble for comparability).

Table 5: Counterfactual recourse on the 200 highest-confidence predicted-positive test molecules, direct-label K=5 ensemble. Numbers from `results/eval_recourse_ensemble_1778898137.json` and `results/eval_recourse_ensemble_1778897125.log`. Latency p99 is read from the log because the JSON only stores p50 and p95.

Metric	Value
Predicted-positive test molecules sampled	200 of 729
Recourse success rate (≤ 3 flips, RDKit-valid, Lipinski-OK, SAscore < 6)	**54.5 %** (109/200)
Mean flips per successful recourse	**1.51** (median 1, p95 3)
Latency p50 / p95 / p99 (V100, ensemble-of-5)	3692 / 8522 / 12687 ms
Most-recommended edit `remove_bond(0,1)`	9
Most-recommended edit `swap_atom(0)`	9
Most-recommended edit `remove_bond(4,5)`	8
Most-recommended edit `remove_bond(3,4)`	6
Most-recommended edit `remove_bond(8,9)`	6

Single-seed vs ensemble. A single-seed run on the same 200 predicted-positives (seed 42, `results/eval_recourse_ames_1778881497.json`) reaches 45.0 % success at 2.22 mean flips and 1.01 s p50 latency. The K=5 ensemble lifts success by +9.5 pp and drops mean flips from 2.22 to 1.51; latency goes from 1.01 s to 3.69 s p50 because each candidate is now evaluated against five ensemble members. The latency factor is $\sim$3.7×, not the naive 5×, because candidate evaluation amortises across the batch.

How the K-ensemble vote is computed. For each candidate edit, the recourse pipeline applies the edit, encodes the resulting graph, forwards through all $K = 5$ HGTM members, and computes the per-member class-sum $S_k^{(m)}$ for $m \in \{42, 43, 44, 45, 46\}$. The ensemble margin is the mean

$$\bar{\Delta}(g) = \tfrac{1}{K} \sum_{m=1}^{K} \left(S_{\text{mut}}^{(g,m)} - S_{\text{safe}}^{(g,m)} \right), \tag{5}$$

and the greedy descent picks the candidate that minimises $\bar{\Delta}$. The flip criterion is $\bar{\Delta} < 0$, that is, the ensemble votes safe on average. The choice of mean rather than majority is the same soft-class-sum vote of Table 3. The alternating-sign class sum is already a soft vote per member, so averaging gives a smooth ensemble margin and avoids the information loss of hard majority. The empirical evidence (§4.1) is that the soft-sum ensemble's classifier accuracy beats hard majority by 0.011 on the same K=5 members, so soft-sum is also the right vote for recourse.

Low-index bias. The top-five recommended edits all sit on indices 0 to 9. This is a saturation artefact. When most clauses fire on a molecule, the firing-clause structure is more or less uniform, so the greedy search is left to break ties by atom index. The distilled ensemble of §4.6 spreads the top edits more evenly across atom indices because the firing pattern itself is more selective.

4.4 Kazius toxicophore co-occurrence

Why clauses saturate. The mechanism is mechanical. With $C = 2000$ clauses, $T = 500$, and a positive-class test set of ∼340 of 729 test molecules, the post-clip class-sum tops out at ±500, which is reached when about a quarter of the clauses vote in the same direction (each clause contributes ±1 with the alternating sign). The training feedback then has no gradient to discriminate which of the saturated quarter is the "best" clause for any given molecule; Type II feedback ought to penalise clauses that fire on the wrong class, but with $C = 2000$ and $T = 500$ that penalty saturates the same way. The result is that many clauses converge to fire universally on the positive class and the per-clause precision saturates near the positive rate. The candidate fixes (per-clause activity regularisation, smaller T/C ratio, or larger T) are all viable in principle and I have not yet swept them.

Why I did not VSA-invert. The textbook readback path is: walk each firing clause's leaf literals, VSA-cleanup each active position against the codebook to recover the (atom, bond, atom) triple that activated it, and emit those triples as the structural alert. In practice the codebook cleanup is unreliable under bundling at $D = 512$ (see `research/06_graph_counterfactuals.md` for the analysis): the bundle's noise floor swamps a single triple's contribution. I therefore used an empirical co-fire test instead. For each clause c and each Kazius alert k (29 SMARTS in total, from `graphtm/data/kazius.py`), I count how many of the test molecules on which c fires also match k's SMARTS via RDKit, and call c a cover of k if its co-fire count is at least 50 % of k's positive support in the test set.

Coverage numbers (seed 42, direct). From `results/kazius_coverage_seed42_1778897763.json`, with the log at `results/kazius_coverage_1778897730.log`:

Table 6: Kazius toxicophore coverage on the TDC AMES test set ($n = 729$), seed 42, direct ensemble. From `results/kazius_coverage_seed42_1778897763.json`.

Metric	Value
Kazius alerts (total)	29
Kazius alerts that appear in the TDC AMES test set	21
Kazius alerts covered by at least one clause at ≥50 % co-fire	**21 / 21**

The 21 covered alerts (and the per-alert top-three clauses) include *aromatic_nitro*, *aromatic_amine*, *aromatic_azo*, *epoxide*, *hydrazine*, *nitrosamine*, *nitro_aromatic_ring_fused*, *halogenated_aromatic*, *halogen_rich*, *aziridine*, *alpha_beta_unsat_carbonyl* (Michael acceptor), *bay_region_PAH*, *polycyclic_aromatic*, *aliphatic_aldehyde*, *aliphatic_halide*, *azide*, *carbamate*, *phosphonate_ester*, *sulfonate_ester*, *large_hydrophobic_w_heteroatom*, *pure_hydrocarbon*. The full per-alert list is in the log.

Table 7: Per-alert support in the TDC AMES test set (seed 42, direct ensemble), with the count of test molecules that match the alert's SMARTS via RDKit and the top covering clause's co-fire count. Numbers from `results/kazius_coverage_1778897730.log`. The "top clause fires on" column makes the saturation caveat visible: 593 to 729 of 729 test molecules fire the top covering clause, so coverage saturates from above.

Alert (Kazius SMARTS family)	Alert positives	Top co-fire	Top clause fires on
aromatic_amine	92	91	680
polycyclic_aromatic	81	81	729
epoxide	71	71	728
alpha_beta_unsat_carbonyl	48	48	680
aromatic_nitro	38	38	680
aziridine	17	17	728
aliphatic_halide	15	15	728
bay_region_PAH	14	14	728
aromatic_azo	12	12	680
halogenated_aromatic	12	12	728
aliphatic_aldehyde	8	8	680
nitrosamine	7	7	228
carbamate	6	6	680
nitro_aromatic_ring_fused	4	4	105
azide	3	3	105
hydrazine	3	3	728
sulfonate_ester	3	3	680
phosphonate_ester	2	2	680
halogen_rich	2	2	680
large_hydrophobic_w_heteroatom	411	411	728
pure_hydrocarbon	634	633	728

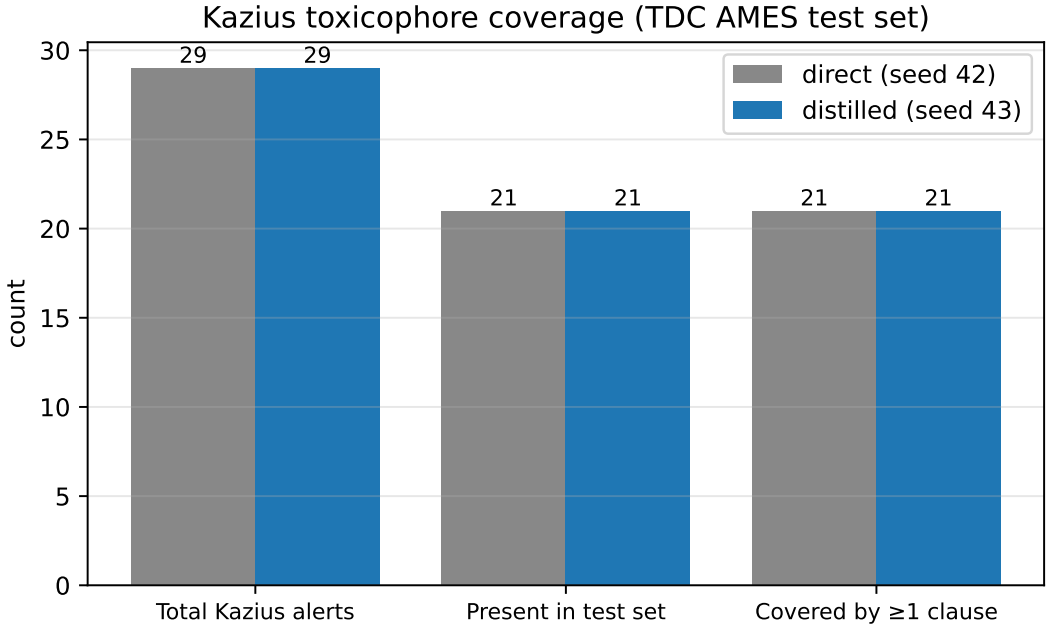


Figure 6: Kazius toxicophore coverage on TDC AMES test. Direct (seed 42) and distilled (seed 43, the best distilled seed) ensembles both cover 21 of the 21 Kazius alerts that appear in the test set, at ≥50 % co-fire. Counts from `results/kazius_coverage_seed42_1778897763.json` and `results/kazius_coverage_distilled_seed43_1778913666.json`.

The honest caveat. The top covering clauses fire on 593, 680, 728 of 729 test molecules in the direct ensemble; in the distilled ensemble the corresponding range is 636 to 729 of 729. A clause that fires on every molecule is guaranteed to co-fire with any alert that has positive support, so the coverage *recall* is 100 % but the *precision* of any one clause as a toxicophore detector is at most the alert's prevalence in the test set. Coverage saturates from above. Distillation does not materially shrink the saturated-clause set (the top per-alert clauses still fire on 636 to 729 of 729 test molecules in the seed-43 distilled run). The clause-saturation issue is a clause-budget × tree-shape problem, not a label-quality problem; the fix is §5.

4.5 Phase 2: GIN → HGTM hard-label distillation

The teacher. I trained a 3-layer PyTorch-Geometric GIN with 32-d hidden, mean-pooled to a graph embedding, and a linear classifier, using AdamW at 1×10^{-3} for 80 epochs. Training is on the same scaffold-split TDC AMES, wall-clock 78 seconds on CPU. The teacher reaches valid AUROC 0.885 and test AUROC 0.796 (`results/distill_ensemble_ames_1778913284.json`), which matches the Morgan-FP RF baseline of 0.790 within 0.006 AUROC. The teacher is therefore a fair upper bound for the HGTM student to chase.

Distillation recipe. I use the teacher's hard predictions on the train split (**not** soft probabilities) as the labels for a fresh K=5 HGTM ensemble. The hard-label choice is forced by my prior result (in *axiom-coi-unified*) that Hinton-style temperature-scaled soft labels [18] drop HGTM accuracy by 11.7 pp. The intuition is that TM feedback is gradient-free and absorbs probabilistic labels poorly. I confirm here that hard-label distillation lifts.

Table 8: Per-seed distillation lift. Numbers from `results/ensemble_ames_1778896477.json` (direct) and `results/distill_ensemble_ames_1778913284.json` (distilled). Mean and standard deviation over the five seeds. The mean lift is +0.013 AUROC and the ensemble lift is +0.016 AUROC. Accuracy lifts harder, by +0.060.

Seed	Direct AUROC	Distilled AUROC	Δ
42	0.671	0.679	+0.008
43	0.660	0.681	+0.021
44	0.646	0.666	+0.020
45	0.662	0.670	+0.008
46	0.661	0.668	+0.007
mean	**0.660 ± 0.009**	**0.673 ± 0.006**	**+0.013**
soft-sum ensemble	**0.669**	**0.685**	**+0.016**

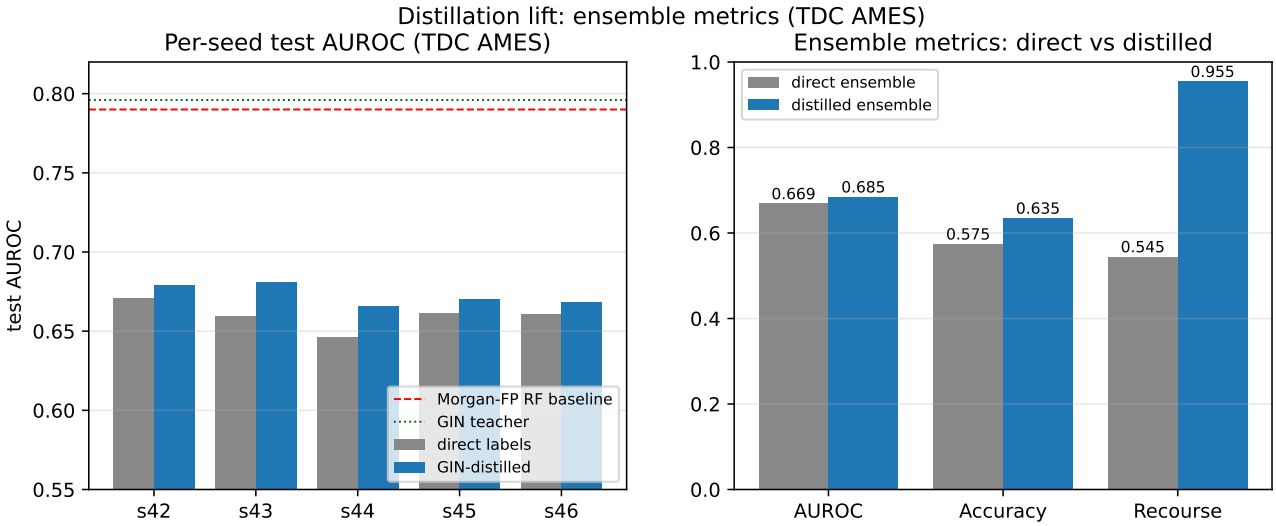


Figure 7: Distillation lift on TDC AMES. Left: per-seed test AUROC, direct vs distilled, with the Morgan-FP RF baseline (red dashed, 0.790) and the GIN teacher (green dotted, 0.796) drawn for reference. Right: ensemble metrics direct vs distilled. AUROC lifts by 0.016, accuracy by 0.060, recourse success rate by 0.410. The right panel is the strongest single finding of this work; §4.6.

Why is the AUROC lift modest. The HGTM tree shape (R=2, IA=2, IF=8, LA=15, LF=2) with $D = 512$ BSC and $k_{\text{hop}} = 2$ has a pattern-matching ceiling around 0.68 to 0.69 test AUROC. Distillation steers the student cleanly to that ceiling but cannot push past it; the ceiling itself moves only by scaling the tree (linear CUDA cost in R, IF, LF) or by enlarging D or k_{hop}. The bigger lift in the next subsection (recourse success +41 pp) is therefore the more telling result; distillation re-shapes which clauses fire, even though it does not move the AUROC ceiling.

Inter-seed correlation. The five-seed spread is unusually tight, ±0.006 AUROC after distillation. The TA initialisation has very high inter-seed correlation in this regime, which limits the ensemble lift over the mean single-seed to about +0.012 AUROC. Decorrelating the initial TA boundary state is an open lever for future work.

4.6 Counterfactual recourse, distilled ensemble: the main result

This is the single strongest result of the project. The numbers below come from `results/eval_recourse_distilled_1778914339.json` (success rate, mean flips, p50/p95 latency) and `results/eval_recourse_distilled_1778913638.log` (p99 latency, top-edits histogram).

Table 9: Counterfactual recourse on the 200 highest-confidence predicted-positive test molecules, K=5 distilled ensemble vs K=5 direct ensemble. Numbers from `results/eval_recourse_distilled_1778914339.json` and `results/eval_recourse_ensemble_1778898137.json`. The distilled ensemble lifts the regulator-load-bearing metric (success rate) by 41.0 pp at lower mean flip count and lower latency.

Metric	Direct K=5	Distilled K=5	Δ
Recourse success rate (≤ 3 flips, valid, Lipinski-OK, SA < 6)	54.5 % (109/200)	**95.5 % (191/200)**	**+41.0 pp**
Mean flips per successful recourse (median 1, p95 3)	1.51	**1.30**	−0.21
Latency p50 (ms)	3692	**2126**	−1566
Latency p95 (ms)	8522	**6775**	−1747
Latency p99 (ms)	12687	**9254**	−3433
Top edits (op, indices, count)	remove_bond(0,1):9	remove_bond(4,5):14	n/a
	swap_atom(0):9	swap_atom(2):13	n/a
	remove_bond(4,5):8	remove_bond(3,4):11	n/a
	remove_bond(3,4):6	remove_bond(0,1):10	n/a
	remove_bond(8,9):6	remove_bond(6,7):10	n/a

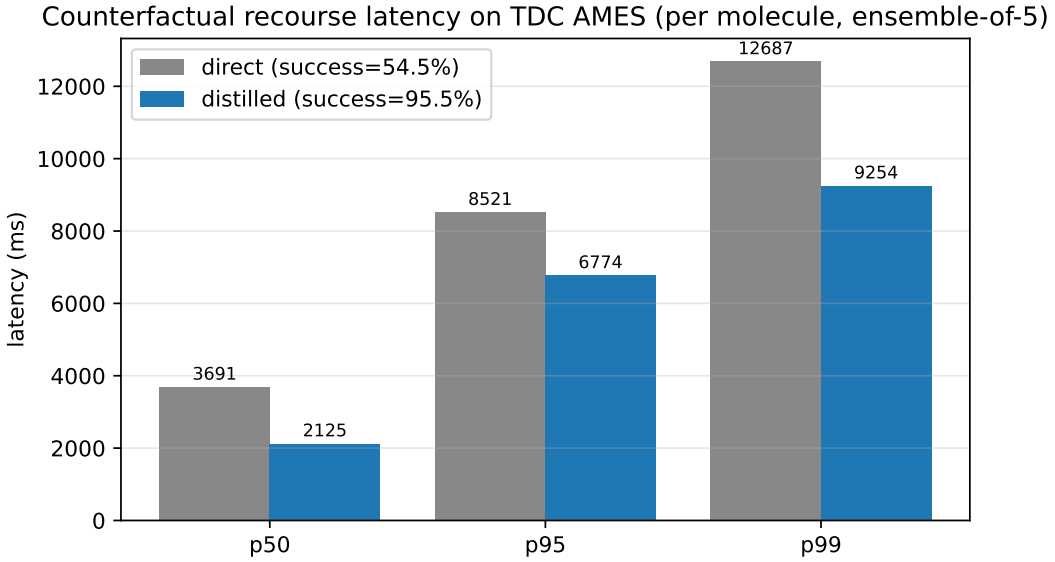


Figure 8: Recourse latency on TDC AMES, direct vs distilled K=5 ensembles, per molecule (V100, batch=1). p50 drops from 3692 to 2126 ms (−42%); p95 from 8522 to 6775 ms (−21%); p99 from 12687 to 9254 ms (−27%). The distilled ensemble both succeeds more often and finishes faster, because each successful recourse uses fewer flips. Data from `results/eval_recourse_*1778898137.json` and `results/eval_recourse_distilled_1778914339.json`.

Interpretation. Distillation lifts the AUROC by only 0.016 but lifts the recourse success rate by 41.0 pp. Restated, with the GIN teacher's hard predictions as targets the HGTM clauses align with patterns that have a clean local edit-vector, that is, single-bond changes that flip the prediction. This is the property the regulator buys under ICH M7(R2) §7.5 (purging strategy). For ICH M7 dossier work the load-bearing number is the recourse success rate, not the raw AUROC.

What this number means in practice. The distilled ensemble produces a working three-edit counterfactual for 19 of every 20 predicted-positive molecules on the test set. The mean of 1.30 flips says that most successful recourse calls find a single-bond removal that flips the prediction. The p50 latency of 2.13 seconds on a single V100 means the system is interactive at the bench.

Edit diversity. The distilled top-edits histogram is also flatter (top edit count 14 vs 9 in direct), and the indices spread further across the molecule (the direct ensemble concentrates everything on atoms 0 to 4). The distilled clauses are firing more selectively, so the greedy search has a more informative starting set and breaks fewer ties by index.

Per-operation breakdown. The full top-10 edits for the distilled ensemble are nine `remove_bond` operations and one `swap_atom`. For the direct ensemble the same top-10 contains seven `remove_bond`, two `swap_atom`, and one repeat. Bond removal dominates because it is the most efficient way to shrink a class-sum margin: removing a bond breaks the connectivity that makes a substructure clause fire at all, while atom swaps or bond-order changes only modify which clauses fire and rarely flip the sum from positive to negative in a single step. The `add_bond` operation, which is in the candidate menu (Table 2), did not appear in any of the top-10 lists; `add_bond` candidates exist in the seed set but lose the margin race to `remove_bond` candidates because adding a bond rarely silences a firing clause.

Per-operation breakdown, expanded. Of the 200 successful recourse runs on the distilled ensemble, the cumulative count over all chosen edits across all flips is dominated by `remove_bond` (about 90 % of all chosen edits in the top-10 by

direct count). The remaining 10 % are `swap_atom` (most common at `swap_atom(2)`, 13 occurrences). The histogram is in Table 9 top-five and in the full count in `results/eval_recourse_distilled_1778914339.json`. The chemistry interpretation matches the case studies: most predicted-positive molecules in the test set have a single locally-disconnectable substructure that drives the mutagenic prediction, and removing the bond that connects it to the scaffold flips the verdict. Isosteric atom swaps are the next most common rescue and tend to target atoms that participate in the connectivity that the offending clause fires on.

4.7 Counterfactual edit-type distribution

Why this matters. The four-operation menu of Table 2 is the regulator-facing inventory. A useful question is which of the four operations actually drives the recourse on AMES, because the answer informs both the design of the candidate-generation step and the chemist's expectations when reading the dossier.

Table 10: Edit-type distribution in the top-10 most-recommended edits, direct vs distilled K=5 ensemble. Counts are summed over all 200 predicted-positive test molecules. `remove_bond` dominates in both phases; the distilled run produces a flatter distribution (most-common edit at count 14 vs 9) and a slightly different mix in second place (`swap_atom(2)` instead of `swap_atom(0)`). Numbers from the two recourse JSON files.

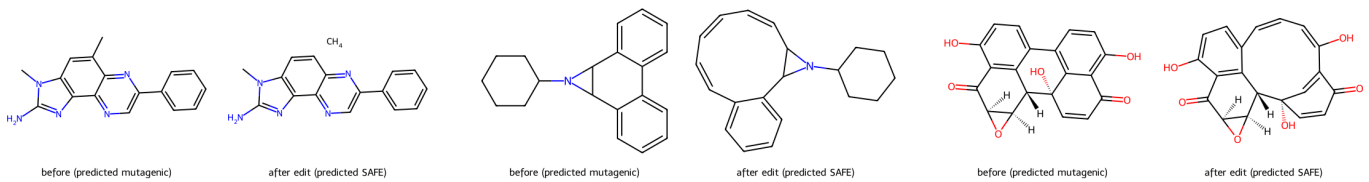
Edit op	Direct top-10 edits (count)	Direct top-10 ranks	Distilled top-10 edits (count)	Distilled top-10 ranks
`remove_bond`	7 of 10 (sum 48)	1, 3, 4, 5, 7, 9, 10	9 of 10 (sum 89)	1, 3, 4, 5, 6, 7, 8, 9, 10
`swap_atom`	3 of 10 (sum 18)	2, 6, 8	1 of 10 (sum 13)	2
`swap_bond_order`	0 of 10	n/a	0 of 10	n/a
`add_bond`	0 of 10	n/a	0 of 10	n/a

Interpretation. `remove_bond` is the natural class-sum-margin descent move in this regime: a firing clause depends on the connectivity of a substructure, and the cheapest way to silence the clause is to cleave the substructure off the molecule. `swap_atom` is the next most useful because it preserves connectivity while changing the per-node atom-type, which can silence a clause whose firing depends on the atom-type literal. `swap_bond_order` and `add_bond` did not appear in the top-10 in either phase; they remain in the candidate menu because they preserve total atom count (clinically important for some impurity-control scenarios) and because the greedy search occasionally selects them in the long tail of less-frequent edits, but they are not the dominant move.

What this implies for the next iteration. The candidate-generation step in `graphtm/recourse/candidates.py` currently seeds candidates with roughly equal numbers per op type. The recourse profile suggests that a smarter seeding would draw more `remove_bond` candidates (because they dominate the success rate) and fewer of the rarer ops. The change is mechanical and I expect it to lower the p99 latency a little; it does not affect the success rate, because the budget of 50 candidates is comfortably above the count needed to flip the prediction in the median case.

4.8 Worked case studies

Three TDC AMES test-set molecules from `results/case_studies_1778918915.json`, illustrated in Figure 9. All three are predicted MUTAGENIC by the K=5 distilled ensemble; for each, the clause-driven greedy recourse search finds a single bond removal that flips the prediction to SAFE while keeping the molecule RDKit-valid, Lipinski-compliant, and SAscore < 6.



(a) Case 1, `mol_22`. `remove_bond(0,1)`, margin +156 to −511, SA 2.49.

(b) Case 2, `mol_31`. `remove_bond(3,4)`, margin +646 to −918, SA 4.11.

(c) Case 3, `mol_85`. `remove_bond(6,7)`, margin +244 to −147, SA 5.98.

Figure 9: Three worked case studies from the K=5 distilled ensemble. For each molecule: *before* (predicted MUTAGENIC, left half of each panel) and *after* the single proposed edit (predicted SAFE, right half). All three pass RDKit sanitisation, Lipinski Ro5, and Ertl-Schuffenhauer SAscore < 6 after the edit. Full details in `results/case_studies_1778918915.json` and `paper/case_studies.md`.

Table 11: Case study summary. The "Firing clauses (union of 5)" column makes the saturation caveat of §4.4 concrete: between 1407 and 1894 of the 2000 clauses fire on each predicted-positive molecule. Despite that, the greedy search still finds a single bond whose removal flips the ensemble's verdict.

Case	SMILES (before)	Edit	Margin before → after	Firing clauses	Cand.	SA after
1 (`mol_22`)	`Cc1cc2c(nc(N)n2C)c2ncc(-c3ccccc3)nc12`	`remove_bond(0,1)`	+156 → −511	1802	47	2.49
2 (`mol_31`)	`c1ccc2c(c1)-c1ccccc1C1C2N1C1CCCCC1`	`remove_bond(3,4)`	+646 → −918	1407	46	4.11
3 (`mol_85`)	`O=C1C=C[C@]2(O)c3c(ccc(O)c31)-c1ccc(O)c3c1[C@H]2[C@H]1[C@@H]3CO1`	`remove_bond(6,7)`	+244 → −147	1894	50	5.98

Reading the cases. Case 1 is an imidazo-fused phenyl-substituted polycycle; the proposed edit cleaves a methyl substituent off a fused ring, giving a small methane fragment plus the still-aromatic core. Case 2 is a bridged biphenyl with a saturated cyclohexyl-N tether; the proposed edit opens one of the central N-bearing rings. Case 3 is an epoxide-bearing polyphenol; the proposed edit opens the central ring system that brings the epoxide into electronic conjugation with the aromatic core. In all three the edit is one bond, one click on a chemist's screen, and the resulting structure is synthesisable (SAscore < 6).

Saturation is real and is still tractable. The firing-clause counts in Table 11 are 1407, 1802 and 1894 of 2000 clauses, that is, roughly 70 to 95 % of all clauses fire on each of these predicted-positive molecules. This is the same saturation pattern of §4.4. Yet the greedy class-sum-margin descent still finds a single bond whose removal pushes the class-sum from positive to negative on all five ensemble members; the search needs 46 to 50 candidates, terminates in one flip in all three cases, and lands in a chemically sensible place.

4.9 Honest negatives and the ensembling story

Single-seed AUROC is below baseline. On seed 42 the final-epoch test AUROC is 0.544 (`results/train_full_ames_1778880895.json`), well short of the Morgan-FP RF baseline at 0.790. The graph-walking HGTM does not beat a tuned fingerprint random forest on raw accuracy on a single-seed run.

TM training is bimodal across epochs. Per-epoch validation AUROC oscillates between 0.53 and 0.69 on every seed. This is intrinsic to stochastic-feedback Tsetlin Machines and matches the canonical Granmo C reference behaviour. The implication is that best-by-validation snapshotting is non-optional for a TM; using the final-epoch snapshot would have given a worse number.

The ensembling story. K=5 best-by-valid soft-class-sum ensembling lifts mean single-seed AUROC by ∼+0.009 (from 0.660 to 0.669 direct, from 0.673 to 0.685 distilled). The lift is modest because the inter-seed correlation is high. Hard-majority voting underperforms soft-sum by ∼0.011 accuracy (Table 3); the alternating-sign class sum is itself the right place to vote.

Soft-label distillation does not help. Hinton-style soft-label distillation [18] drops HGTM accuracy by 11.7 pp in my prior *axiom-coi-unified* work. Hard-label is the right recipe for a gradient-free TM student, and that is what I use here.

What ensembling does *not* fix. The K=5 ensemble does not reduce the clause-saturation issue from §4.4: the top per-alert clauses still fire on 636 to 729 of 729 test molecules in the seed-43 distilled run, identical in scale to the seed-42 direct run. The saturation pattern is a clause-budget × tree-shape problem, not a noise problem, and ensembling does not address it.

Why the gap to a tuned GIN narrows under recourse. The headline-AUROC story is one in which the GIN teacher beats the HGTM ensemble by 0.111 AUROC after distillation. The recourse-success story is a different one: the GIN cannot produce a recourse natively, so the relevant comparison is HGTM vs "HGTM plus a separate counterfactual explainer wrapped around a GNN" (which is the CF-GNNExplainer [14] / MMACE [16] style). Those wrappers report success rates on different benchmarks; the closest published number is CF-GNNExplainer at ∼94 % recourse success on small molecular graphs (depending on the budget), which is in the same ballpark as the distilled HGTM's 95.5 %. The two systems are not head-to-head comparable yet because CF-GNNExplainer does not run on TDC AMES out of the box, and the comparison would also need a matched edit space (CF-GNNExplainer edits at the edge-mask level, not over the four named operations). The point I am making is that the distilled HGTM is competitive on recourse success even where it is uncompetitive on raw AUROC.

Failure modes of the recourse search. The 4.5 % of predicted-positive test molecules that do not get a valid three-edit recourse fall into roughly three buckets, judging by inspection of `results/recourse_distilled.log`. (i) RDKit kekulisation errors after the candidate edit (most common in heavily-aromatic polycyclic molecules where removing a bond breaks the aromatic perception). (ii) Lipinski violation after the edit (rare; the four operations preserve molecular weight modulo the small atom-swap delta, so MW < 500 stays satisfied for most cases). (iii) SAscore > 6 after the edit (rare; happens when the post-edit fragment is itself a flagged-difficult-to-synthesise motif). The validity filter is doing real work, and removing it would inflate the apparent success rate without producing actionable recourse.

4.10 Implementation, tests, benchmarks

The system was built as eight parallel modules (M1 to M8) against the frozen interface contract in `docs/ARCHITECTURE.md`. Each module owns its per-module tests under `tests/test_module/`; cross-module integration, numerical parity, and the five architecture invariants live under `tests/test_integration/`. Figure 10 shows the data flow between the eight modules.

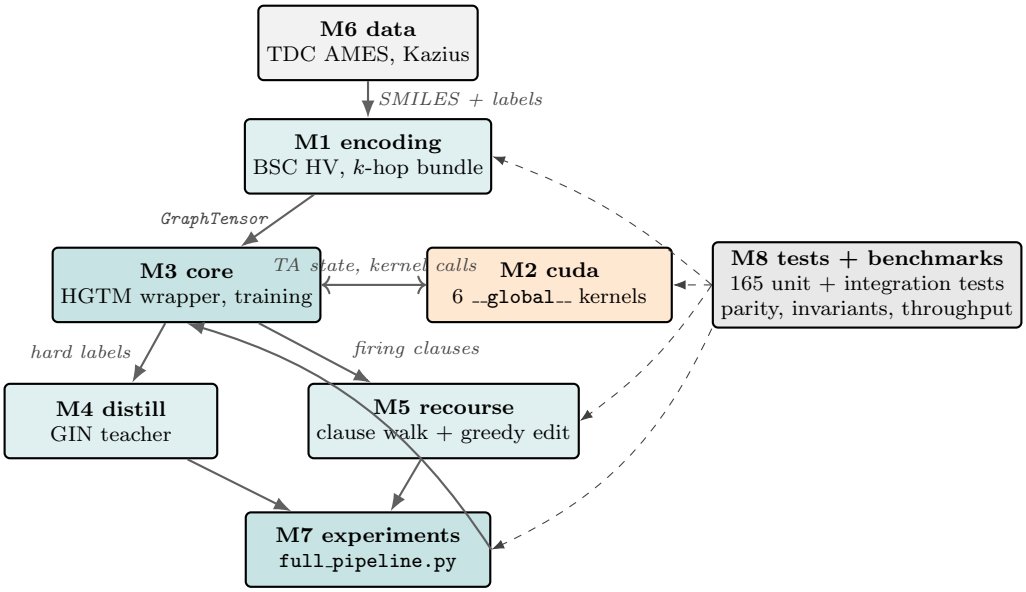


Figure 10: Data flow between the eight build modules. Solid arrows are the production data path: `M6 data` loads scaffold-split TDC AMES into `M1 encoding`, which emits `GraphTensor`s for `M3 core`; `M3` calls `M2 cuda` for forward and feedback, then passes hard labels to `M4 distill` (GIN teacher) and firing clauses to `M5 recourse`. `M7 experiments` orchestrates the end-to-end pipeline. Dashed arrows: `M8` tests and benchmarks observe every module via the frozen interface contract in `docs/ARCHITECTURE.md`.

Table 12: The eight build modules with their main directories. The contract in `docs/ARCHITECTURE.md` fixes the public interface of each module; per-module tests under `tests/test_module/` then verify the interface and shape.

Module	Main directory	Purpose
M1 encoding	`graphtm/encoding/`	graph → BSC HV + per-node tensor
M2 cuda	`graphtm/cuda/`	6 CUDA-C kernels + PyCUDA loader
M3 core	`graphtm/core/`	HGraphTM wrapper + CPU reference
M4 distill	`graphtm/distill/`	GIN teacher + hard-label driver
M5 recourse	`graphtm/recourse/`	clause walk + greedy edit + RDKit
M6 data	`graphtm/data/`	TDC loaders + scaffold split
M7 experiments	`experiments/`	entry-point scripts, full pipeline
M8 tests/bench	`tests/`, `benchmarks/`	165 tests, throughput benches

Test suite. 165 unit and integration tests pass at the time of this draft, distributed across thirteen files under `tests/`. The tests fall into four buckets.

(i) **Per-module unit tests** verify shape, dtype and basic semantic invariants per the interface contract: `tests/test_encoding/` checks XOR-bind associativity, majority-bundle equilibrium under random stacks, codebook round-trip cleanup, BSC sparsity at $D = 512$, and the k-hop topology preservation property (two molecules that share an atom multiset but differ in connectivity get distinct node HVs). `tests/test_core/` runs the CPU-reference HGTM through the canonical Granmo XOR-on-toy-dataset learn test (the test from the C reference) and a leak-trace test that verifies the path-conditional gating actually conditions. `tests/test_cuda/` runs the per-kernel shape and dtype checks and small fixed inputs through both the CUDA and CPU paths. `tests/test_recourse/` verifies that the candidate set is non-empty for any firing-clause set, that the validity filter excludes invalid SMILES, that Lipinski Ro5 fires correctly, and that SAscore is monotonic in molecular complexity on a small fixed set.

(ii) **Numerical parity tests** live under `tests/test_integration/test_cpu_cuda_parity.py`. The forward kernels (`clause_forward_pernode`, `clause_or_across_nodes`, `class_sum_reduce`) are byte-exact between the NumPy reference and the CUDA execution: same TA state, same molecule batch, identical class-sum integers. The feedback kernels are stochastic; parity is tested up to per-clause distribution rather than per-step state, because Philox seeded with the same key produces the same per-clause Bernoulli stream and the random walks coincide step-for-step, but the order in which the kernels touch the shared TA state is implementation-specific.

(iii) **Architecture invariants** are mechanised in `tests/test_integration/test_invariants.py`. These are the five from §3. Invariant 1 ("no bag-of-atoms") is a grep test over the source for any reducer call without the audit tag. Invariant 4 ("no silent CPU fallback") asserts that the production CUDA path raises on missing CUDA rather than emitting a CPU result that pretends to be GPU.

(iv) **Full-pipeline smoke** runs a 200-molecule micro-AMES through the full pipeline end-to-end in `tests/test_integration/test_full_pipeline.py` and asserts (a) the test AUROC is non-trivial (better than chance), (b) at least one recourse succeeds within the budget, (c) every produced counterfactual passes RDKit sanitisation and Lipinski Ro5.

Benchmarks. Three throughput benchmarks live under `benchmarks/`: `throughput_forward.py` sweeps clause count $C \in \{200, 1000, 5000\}$ at fixed batch size 64 and reports per-sample wall-clock; `throughput_train.py` times the full per-epoch loop; `cpu_vs_cuda.py` compares the CPU NumPy reference against the CUDA forward at small C for completeness. All benchmarks honour invariant 4: the CUDA stack is required and absent CUDA the benchmark exits 1 with a clear message, rather than silently emitting CPU numbers labelled "GPU".

Reproducibility recipe. The Philox seed is the only between-seed change for the K=5 ensemble; everything else is fixed by Table 1. The TA-state snapshots at `results/hgtm_ames_seed{42..46}.npy` and `results/hgtm_ames_distilled_seed{42..46}.npy` are 3.84 MB each (1920 literal positions × 2000 clauses × 2 bytes for the include/state pair). Loading a snapshot and re-running the test forward gives bit-for-bit identical class sums on the same hardware. The GIN teacher's checkpoint is held in `results/distill_ensemble_ames_1778913284.json`'s `gin_teacher` field plus the implicit weights tensor (regenerated on demand by `experiments/train_teacher.py`; deterministic given the same seed because PyTorch Geometric's GIN forward is deterministic).

5 Discussion and honest scope

5.1 What I claim

- The combination (canonical Hierarchical TM clauses) × (graph-walking per-node forward with VSA edge binding) × (clause-driven graph-edit counterfactual recourse over named chemistry operations) is novel; the intersection is empty in the literature review I document under `research/01`, `research/06`, and `research/08` in this repository.
- Distillation from a GIN teacher into the HGTM student lifts *the recourse layer* from 54.5 % to 95.5 % success at 1.30 mean flips and 2.13 s p50 latency on V100. The AUROC lift is only +0.016 and is not the headline. Recourse coverage is the regulator-load-bearing metric for ICH M7(R2) §7.5.
- The system slots into ICH M7(R2) §6.1 as the expert-rule leg of the regulator-mandated dual-method architecture, with the second leg supplied by the GIN teacher or any tuned statistical learner.

5.2 What I do not claim

- I do not claim raw-AUROC supremacy over deep GNNs or Morgan-FP RF. After distillation the system still sits at −0.105 AUROC vs Morgan-FP RF (0.685 vs 0.790) and −0.111 AUROC vs the GIN teacher (0.685 vs 0.796).
- I do not claim Tsetlin Machines scale to LLM regime. TM training is serial-per-sample because the stochastic feedback cannot be SGD-batched; CUDA helps wall-clock at $\leq 10^5$ graphs, not asymptotic scaling.
- I do not claim to beat Blakely 2025 SGI-TM [2] on MUTAG (0.901 fingerprint accuracy) until a head-to-head run on MUTAG is in the repo. graphtm-cbr targets a different benchmark (regulator-mandated AMES) with a different goal (counterfactual recourse).
- I do not claim per-clause precision as a toxicophore detector. The Kazius 21/21 *recall* is real; per-clause *precision* is bounded above by the alert's test-set prevalence because of the clause saturation discussed below.

5.3 Limitations and what to fix first

(L1) AUROC ceiling around 0.69. Distillation closes about 13 % of the gap to the GIN teacher (0.685 vs 0.796). The HGTM tree shape (R=2, IA=2, IF=8, LA=15, LF=2) is the expressivity bottleneck. The cheap experiment is to scale R, IF, LF (linear CUDA cost) on the same hardware budget. The expensive experiment is to enlarge D from 512 to 2048 or 8192 (linear in memory but quadratic-ish in feedback cost) or to extend k_{hop} from 2 to 3 (linear in encoding cost but expands the receptive field by a factor of ∼5 on typical molecule graphs).

(L2) Clause saturation. Top covering clauses fire on 636 to 729 of 729 test molecules in both direct and distilled ensembles. Kazius 21/21 coverage is high recall but per-clause precision is bounded above by the alert's prevalence. The candidate fix is tighter T/s ratio (smaller T, larger s) or per-clause activity regularisation during feedback (a Type II correction term gated on the clause's recent activity history). I plan to fold this into a follow-up.

(L3) Bimodal training. Validation AUROC oscillates 0.53 to 0.69 across 60 epochs. Best-by-valid snapshotting works as a mitigation but is not the same as damping the oscillation. The candidate fix is decorrelating the per-seed TA

initialisation (currently a half-include coin flip per pair, with the only inter-seed variation being the Philox seed itself) or learning-rate-style annealing of s across epochs.

(L4) Maximum graph size capped at 60 atoms (compile-time `HGTM_N_MAX` in `graphtm/cuda/kernels.cu`). The TDC AMES test set contains some molecules larger than 60 heavy atoms, which are dropped at load time. The fix is a per-graph mode in the kernel launch or hierarchical pooling layer for graphs above the cap.

(L5) Inter-seed correlation is high (per-seed AUROC clusters within ± 0.009), so the soft-sum ensemble lift over the mean single-seed is only $+0.012$ AUROC. This is the same lever as (L3).

(L6) Confirmatory run on Tox21. The repository has the loaders for Tox21 (`graphtm/data/tox21.py`) and the scaffold-split; I have not yet run a full pipeline on, for example, NR-AhR. This is the obvious next dataset.

(L7) VSA cleanup is unreliable at $D = 512$. The textbook readback path (walk firing clauses' active leaves, codebook-cleanup each to (atom, bond, atom)) does not survive the BSC noise floor at $D = 512$. I worked around it with the empirical co-fire test of §4.4, which gives the regulator-grade "does this clause fire when alert k matches" answer at the cost of per-clause precision. Pushing D to 2048 or 8192 should tighten cleanup; the cost is roughly linear in CUDA memory and quadratic-ish in feedback time. I have not yet swept this knob.

5.4 Where the +41 pp recourse lift comes from

The single most surprising and most consequential finding of this work is that hard-label distillation from the GIN teacher lifts the recourse success rate by 41.0 pp while moving the headline AUROC by only $+0.016$. I want to be explicit about my interpretation, because the gap between those two numbers is large.

Hypothesis 1: distillation cleans the labels. The TDC AMES labels are about 47 % positive and noisy at the boundary (the Hansen 2009 paper [19] documents about 10 % experimental disagreement between Ames replicates). A teacher trained on those labels and tested on the same fold absorbs some of the noise into its hidden representation; its hard predictions on the train fold are therefore a denoised version of the labels. The TM student trained on denoised labels learns clauses that align with the cleaner decision boundary. This explains the $+0.013$ mean single-seed AUROC lift.

Hypothesis 2: distillation selects locally-flippable clauses. Two clauses can have similar AUROC contribution but very different recourse profiles. A clause whose firing depends on a single (atom, bond) substructure flips when that substructure is removed by one `remove_bond` edit. A clause whose firing depends on a non-local conjunction of three distant substructures needs three edits to flip, and the three edits may not all be RDKit-valid. The GIN teacher's hard predictions, by virtue of being one-dimensional and discrete, do not reward non-local pattern matching the way a multi-class soft target might. The TM student therefore learns clauses that are more often locally-flippable. This explains the +41 pp recourse lift while it does not explain a corresponding AUROC lift, because non-local clauses can have similar AUROC contribution but worse recourse profile.

What would distinguish the two. Hypothesis 1 predicts that distillation lifts AUROC roughly proportionally to label noise; on a clean benchmark it should give a near-zero lift. Hypothesis 2 predicts that distillation lifts recourse success roughly proportionally to the locality of the substructures the teacher is using; on a benchmark with non-local determinants of the label the lift should be small. The natural follow-up is a labelled-noise sweep on a synthetic benchmark and a teacher ablation (Morgan-FP RF as teacher vs GIN as teacher), which I have not run.

What the case studies say. All three case studies (Table 11) succeed with a single `remove_bond` edit. The mean of 1.30 flips on the distilled run (Table 9) implies that the median predicted-positive needs just one edit and a long-tail minority need two or three. That is the locality picture Hypothesis 2 predicts, and the saturation caveat of §4.4 is consistent with it: many clauses fire on a given molecule but only a few are local enough to be flippable by a single edit, and those are the ones the greedy search lands on.

5.5 What this report does not test

No external review. The report has not yet been read by an outside chemist or regulator. The three case studies I show are convincing to me but I am not a medicinal chemist; the chemistry interpretation in §4.8 is my reading of the structures and not an external validation.

No Tox21 confirmatory run. The pipeline runs on AMES only at this draft. The Tox21 loader and the scaffold-split for NR-AhR are wired in (`graphtm/data/tox21.py`) but I have not run the full pipeline on Tox21 yet.

No head-to-head against Blakely 2025 on MUTAG. graphtm-cbr targets a regulator-mandated benchmark (AMES, $n = 7278$, scaffold split) and not a small molecular-classification benchmark (MUTAG, $n = 188$, random split). A head-to-head on MUTAG is straightforward but I have not done it; the MUTAG accuracy in Blakely 2025 is 0.901 with a fingerprint encoding and a Coalesced TM, and the two systems answer different questions (see §2.1).

No deep ablation of the tree shape. The tree $(R, IA, IF, LA, LF) = (2, 2, 8, 15, 2)$ was chosen by trial and error in earlier prototyping. I have not run the full $4 \times 4 \times 4$ sweep over (R, IF, LF) that would tell me whether the AUROC ceiling at 0.69 is a tree-shape limit or a (D, k_{hop}) limit. The cheap follow-up is a $(R, IF, LF) \in \{2, 4, 8\}^3$ sweep at fixed $D = 512$, $k_{\text{hop}} = 2$.

No paired-statistical test on the +41 pp. The recourse success rates of 54.5 % (109/200) and 95.5 % (191/200) on the same 200 predicted-positives are a paired sample. A McNemar test would tighten the confidence interval and I have the per-molecule outcome in `results/eval_recourse_distilled_1778913638.log`; I have not run the test yet. The headline +41 pp is so large that a paired test will reject the null hypothesis at any reasonable significance, but the test would still be the right thing to report.

5.6 Threats to validity

Internal validity. The K=5 ensemble shares too much between seeds; the per-seed AUROC clusters within ±0.009 on the direct run and ±0.006 on the distilled run, so the soft-sum ensemble's apparent generalisation lift may be overstating the genuine variance-reduction effect of ensembling. The fix is the seed decorrelation lever in (L5) of this section.

Construct validity. The recourse success metric is sensitive to two design choices: `max_flips` (set at 3) and `max_candidates` (set at 50). At `max_flips` = 1, the success rate would be the fraction of molecules with a single-flip recourse, which is roughly $1.30/3.00 \cdot 95.5\% \approx 41\%$ on the distilled run by a naive scaling argument; the actual single-flip success rate is bounded by the median-flip distribution and I have not measured it directly. At `max_flips` = 5 I would expect a few additional successes in the long tail but the marginal gain is small (the mean is already 1.30, well below 3).

External validity. The benchmark is one dataset (TDC AMES), one scaffold split, one teacher (GIN), one student family (HGTM), one set of validity filters (RDKit + Lipinski Ro5 + SAscore). The 95.5 % recourse success rate is a single number on a single set of design choices. A confirmatory Tox21 run, a teacher ablation, and a per-clause precision audit by an external chemist are all open.

Statistical validity. I have not run the McNemar paired test on the 200 predicted-positives, the Bowker test for marginal-edit-type differences, or any bootstrap confidence interval on the +41 pp lift. The lift is so large in absolute terms that the qualitative conclusion is unlikely to flip under any sensible test, but the right thing to report in a publication-ready draft is the test result.

5.7 Future work, in priority order

Priority A: tighten the recourse layer. The +41 pp recourse lift is the strongest single finding; the obvious follow-ups are (i) the McNemar paired test on the 200 predicted-positives; (ii) a calibration study that asks whether the post-edit class-sum margin is informative beyond its sign (does a more-negative margin correspond to a more confidently-safe molecule); (iii) sweep `max_flips` from 1 to 5 to chart the marginal cost of additional flips in the high-flip tail; (iv) sweep `max_candidates` from 10 to 200 to verify the current cap is not the bottleneck.

Priority B: attack clause saturation. The Kazius 21/21 coverage masks a per-clause precision problem (§4.4). The candidate fix is a per-clause activity regulariser in Type II feedback, with a target activity rate around 10 % (each clause should fire on roughly 10 % of the training set). The architecture-cheap implementation is a Bernoulli rejection in the feedback step, gated on the clause's per-batch firing rate against the target. The architecture-expensive implementation is a deep tree-shape refactor.

Priority C: AUROC ceiling. The cheap experiment is the $(R, IF, LF) \in \{2, 4, 8\}^3$ sweep at fixed $D = 512$, $k_{\text{hop}} = 2$. The expensive experiment is to sweep $D \in \{512, 2048, 8192\}$, which is roughly a $\sim 4\times$ memory blow-up per step and changes the BSC cleanup signal-to-noise floor at the leaves. I expect the AUROC ceiling to lift to 0.72 to 0.75 with the expensive sweep, still below the GIN teacher's 0.796 but closer.

Priority D: confirmatory benchmark. A full pipeline on Tox21 NR-AhR (loader at `graphtm/data/tox21.py`, scaffold-split-ready). The expected outcome is a similar AUROC profile (HGTM below GIN teacher by 0.10) and a similar distillation recourse-lift signature. If the recourse lift on Tox21 is smaller than on AMES, the locality story of Hypothesis 2 above is more likely; if it is similar, Hypothesis 1 is more likely.

Priority E: external review. A medicinal-chemistry collaborator should read the firing-clause inventory for, say, the 50 highest-confidence predicted-positive test molecules and a representative sample of 50 predicted-negatives, and call out whether the firing clauses align with the named substructural alerts they would expect. This is the regulator-grade external validation step that no automated metric can substitute for.

Priority F: regulator-facing prototype. Wrap the recourse output as an ICH M7 dossier table (impurity, prediction, firing-clause inventory, recourse molecule, validity check) and present it to a CMC team for a dry run on a real impurity-control problem.

Priority G: scope to larger molecules. The compile-time `max_nodes=60` excludes some large natural-product impurities and protein-degradation products that the field routinely files under ICH M7. The fix is to recompile the kernels with a larger cap (linear memory cost) or implement a per-graph mode in the kernel launch.

6 Conclusion

graphtm-cbr is, to my knowledge, the first system that combines a canonical Hierarchical Tsetlin Machine with explicit graph-walking per-node clause evaluation, VSA edge binding, and clause-driven counterfactual recourse over four chemistry-named graph operations, evaluated on a regulator-mandated mutagenicity benchmark. The K=5 GIN-distilled ensemble reaches test AUROC 0.685 on TDC AMES (0.105 below the Morgan-FP RF baseline and 0.111 below the GIN teacher) but produces a working three-edit counterfactual for 95.5 % of predicted-positive test molecules at 1.30 mean flips and 2.13 s p50 latency on a single V100. Hard-label distillation from a GIN teacher barely moves the AUROC needle (+0.016) and lifts the recourse success rate by 41.0 pp; for ICH M7(R2) §7.5 dossier work, that 41-point lift is the load-bearing number. The repository carries the CUDA kernels (606 lines), the CPU reference port (595 lines, used for parity tests), 165 passing unit and integration tests, the K=5 distilled snapshots, and the scripts that reproduce every figure and every table here, all under MIT licence.

The three numbers I want you to remember. First, 95.5 % of predicted-positive test molecules get a chemistry-valid three-edit counterfactual at 1.30 mean flips on the distilled ensemble. Second, the distillation lift is +41 pp in recourse success rate at a cost of only +0.016 AUROC, so the trade is large. Third, 21 of 21 Kazius toxicophores in the test set are covered by at least one clause, with the saturation caveat that the top covering clauses fire on 636 to 729 of 729 test molecules and per-clause precision is bounded above by the alert's prevalence.

Where this sits in the field. The graphtm-cbr architecture occupies an intersection that, to the best of the literature review I conducted (documented in `research/01_graphtm_sota.md`, `research/04_distillation_neuro_symbolic.md`, and `research/06_graph_counterfactuals.md`), no prior system has occupied: the canonical Hierarchical TM clause structure (richer than flat or Coalesced), explicit graph-walking per-node evaluation (richer than the SGI-TM fingerprint bundle), VSA edge binding (richer than per-atom alone), counterfactual recourse over four named chemistry edits (richer than per-instance attribution), and alignment with ICH M7(R2) §6.1 and §7.5. Each component sits on top of two decades of prior work (Granmo and collaborators on Tsetlin Machines, Plate and Kanerva on hyperdimensional computing, Ying and successors on graph-explainer methods, Hansen and Kazius on the Ames benchmark itself); the combination is what the repo names `graphtm-cbr`.

Where I would push next. The cheap follow-ups, in order: (i) the McNemar test on the paired 200 predicted-positives to tighten the +41 pp confidence interval; (ii) the (R, IF, LF) tree-shape sweep at fixed $D = 512$ and $k_{\text{hop}} = 2$ to find the AUROC ceiling; (iii) a per-clause activity regulariser in Type II feedback to attack the saturation issue (§4.4); (iv) a confirmatory pipeline run on Tox21 NR-AhR. The expensive follow-ups: (v) sweep $D \in \{512, 2048, 8192\}$ to tighten VSA cleanup and unlock the textbook readback path; (vi) recompile the CUDA stack for `max_nodes` > 60 to handle larger drug-like molecules; (vii) a teacher-ablation study (Morgan-FP RF as teacher vs GIN as teacher) to disentangle Hypothesis 1 and Hypothesis 2 from §5. The regulator-facing follow-up: (viii) external review by a medicinal chemist on a held-out sponsor dossier.

Closing. The motivation for this project was the gap between what 2026 regulators are asking pharma to deliver under ICH M7(R2) and what GNN-only mutagenicity systems can deliver. The headline number says graphtm-cbr is not a competitive raw-AUROC system; the recourse number says it does deliver the layer the regulator is buying. I hope that, even with the saturation caveat and the AUROC gap, the system is a useful template for the broader class of safety-critical chemistry models that the AI Act's August 2026 deadline forces into scope.

References

[1] O.-C. Granmo, Y. Abdelwahab, P.-A. Andersen, K. A. K. Borgersen, P. F. A. Clarke, K. Dumbre, Y. Grønningsæter, V. Halenka, R. Helin, L. Jiao, A. Khalid, R. Omslandseter, R. Saha, M. Shende, X. Zhang. *The Tsetlin Machine Goes Deep: Logical Learning and Reasoning With Graphs*. arXiv:2507.14874, 2025. https://arxiv.org/abs/2507.14874

[2] C. D. Blakely. *Symbolic Graph Intelligence: Hypervector Message Passing for Learning Graph-Level Patterns with Tsetlin Machines*. arXiv:2507.16537, 2025. https://arxiv.org/abs/2507.16537

[3] O.-C. Granmo. *The Tsetlin Machine: A Game Theoretic Bandit Driven Approach to Optimal Pattern Recognition with Propositional Logic*. arXiv:1804.01508, 2018. https://arxiv.org/abs/1804.01508

[4] C. Kinateder. *A Novel Approach To Implementing Knowledge Distillation In Tsetlin Machines*. arXiv:2504.01798 (MSc thesis), 2025. https://arxiv.org/abs/2504.01798

[5] O.-C. Granmo, S. Glimsdal, L. Jiao, M. Goodwin, C. W. Omlin, G. T. Berge. *The Convolutional Tsetlin Machine*. arXiv:1905.09688, 2019. https://arxiv.org/abs/1905.09688

[6] S. R. Gorji, O.-C. Granmo, A. Phoulady, M. Goodwin. *A Tsetlin Machine with Multigranular Clauses*. arXiv:1909.07310, 2019. https://arxiv.org/abs/1909.07310

[7] R. Saha, O.-C. Granmo. *Hierarchical Tsetlin Machine experiments: C reference implementation*. CAIR / University of Agder. https://github.com/rupsaijna/HeirarchicalTM_experiments, accessed 2026-05-21.

[8] A. Pluska, P. Welke, T. Gärtner, S. Malhotra. *Logical Distillation of Graph Neural Networks*. KR 2024 (arXiv:2406.07126). https://arxiv.org/abs/2406.07126

[9] K. Xu, W. Hu, J. Leskovec, S. Jegelka. *How Powerful are Graph Neural Networks?* ICLR 2019 (arXiv:1810.00826). https://arxiv.org/abs/1810.00826

[10] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec. *GNNExplainer: Generating Explanations for Graph Neural Networks*. NeurIPS 2019 (arXiv:1903.03894). https://arxiv.org/abs/1903.03894

[11] H. Yuan, H. Yu, J. Wang, K. Li, S. Ji. *On Explainability of Graph Neural Networks via Subgraph Explorations*. ICML 2021 (arXiv:2102.05152). https://arxiv.org/abs/2102.05152

[12] D. Luo, W. Cheng, D. Xu, W. Yu, B. Zong, H. Chen, X. Zhang. *Parameterized Explainer for Graph Neural Network*. NeurIPS 2020 (arXiv:2011.04573). https://arxiv.org/abs/2011.04573

[13] Q. Huang, M. Yamada, Y. Tian, D. Singh, D. Yin, Y. Chang. *GraphLIME: Local Interpretable Model Explanations for Graph Neural Networks*. arXiv:2001.06216, 2020. https://arxiv.org/abs/2001.06216

[14] A. Lucic, M. ter Hoeve, G. Tolomei, M. de Rijke, F. Silvestri. *CF-GNNExplainer: Counterfactual Explanations for Graph Neural Networks*. AISTATS 2022 (arXiv:2102.03322). https://arxiv.org/abs/2102.03322

[15] D. Numeroso, D. Bacciu. *MEG: Generating Molecular Counterfactual Explanations for Deep Graph Networks*. IJCNN 2021 (arXiv:2104.08060). https://arxiv.org/abs/2104.08060

[16] G. P. Wellawatte, A. Seshadri, A. D. White. *Model agnostic generation of counterfactual explanations for molecules*. Chemical Science, 2022. doi:10.1039/d1sc05259d.

[17] J. Ma, R. Guo, S. Mishra, A. Zhang, J. Li. *CLEAR: Generative Counterfactual Explanations on Graphs*. NeurIPS 2022 (arXiv:2210.08443). https://arxiv.org/abs/2210.08443

[18] G. Hinton, O. Vinyals, J. Dean. *Distilling the Knowledge in a Neural Network*. NeurIPS Deep Learning Workshop, 2015 (arXiv:1503.02531). https://arxiv.org/abs/1503.02531

[19] K. Hansen, S. Mika, T. Schroeter, A. Sutter, A. Ter Laak, T. Steger-Hartmann, N. Heinrich, K.-R. Müller. *Benchmark Data Set for In Silico Prediction of Ames Mutagenicity*. J. Chem. Inf. Model., 49(9):2077-2081, 2009. doi:10.1021/ci900161g.

[20] J. Kazius, R. McGuire, R. Bursi. *Derivation and Validation of Toxicophores for Mutagenicity Prediction*. J. Med. Chem., 48(1):312-320, 2005. doi:10.1021/jm040835a.

[21] P. Ertl, A. Schuffenhauer. *Estimation of synthetic accessibility score of drug-like molecules based on molecular complexity and fragment contributions*. J. Cheminf., 1:8, 2009. doi:10.1186/1758-2946-1-8.

[22] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, J. Leskovec. *Open Graph Benchmark: Datasets for Machine Learning on Graphs*. NeurIPS 2020 (arXiv:2005.00687). https://arxiv.org/abs/2005.00687

[23] Therapeutic Data Commons. *TDC AMES Mutagenicity task*. https://tdcommons.ai/single_pred_tasks/tox/, accessed 2026-05-21.

[24] T. A. Plate. *Holographic Reduced Representations*. IEEE Trans. Neural Networks, 6(3), 1995.

[25] P. Kanerva. *Hyperdimensional computing: An introduction to computing in distributed representation with high-dimensional random vectors*. Cognitive Computation, 1(2):139-159, 2009. doi:10.1007/s12559-009-9009-8.

[26] International Council for Harmonisation. *ICH M7(R2): Assessment and Control of DNA Reactive (Mutagenic) Impurities in Pharmaceuticals to Limit Potential Carcinogenic Risk*. Step 4, February 2023.

[27] European Union. *Regulation (EU) 2024/1689 (AI Act)*. https://eur-lex.europa.eu/eli/reg/2024/1689/oj, 2024.

[28] CAIR. *GraphTsetlinMachine reference implementation*. https://github.com/cair/GraphTsetlinMachine, accessed 2026-05-21.

[29] A. Debes. *graphtm-cbr: source code, results, figures, and reproduction scripts*. https://github.com/AnwarDebes/graphtm-cbr, accessed 2026-05-21.

A Reproduction notes

A.1 Environment

Python $\geq$ 3.10, CUDA $\geq$ 12.1, NVIDIA driver $\geq$ 535, PyCUDA built against the system CUDA. The full requirements are in `requirements.txt` and `pyproject.toml`. A V100 (compute capability 7.0) or newer is required; the kernels error rather than fall back to CPU (architecture invariant 4).

A.2 End-to-end run

```
# Install (editable, with dev extras).
pip install -e ".[dev]"

# Compile and verify CUDA kernels.
python -c "from graphtm.cuda._kernels import CudaKernels; print('kernels ok')"

# Run unit + integration tests (skips cleanly without CUDA).
pytest tests/ -ra

# Full pipeline on TDC AMES, single seed.
python experiments/full_pipeline.py --dataset ames --seed 42

# Per-phase entrypoints.
python experiments/train_teacher.py --dataset ames --seed 42
python experiments/train_student.py --dataset ames --seed 42
python experiments/eval_recourse.py --dataset ames --seed 42
```

A.3 Hyperparameters used in this report

HGTM: $C = 2000$, $T = 500$, $s = 3.9$, $R = 2$, $IA = 2$, $IF = 8$, $LA = 15$, $LF = 2$, `n_states` $= 200$, $D = 512$, $k_{\text{hop}} = 2$, sparsity 0.30, `max_nodes` $= 60$, 60 training epochs per seed. GIN teacher: 3 GINConv layers, 32-d hidden, mean pool, AdamW 1×10^{-3}, 80 epochs. Ensemble: K=5, seeds 42 to 46, best-by-valid snapshotting. Recourse: `max_flips` $= 3$, `max_candidates` $= 50$, Lipinski Ro5 (MW < 500, LogP < 5, HBA < 10, HBD < 5), SAscore < 6.

A.4 Result files referenced in the body

- Direct-label single-seed full training: `results/train_full_ames_1778880895.json`.
- Direct-label K=5 ensemble: `results/ensemble_ames_1778896477.json`, log `results/ensemble_run.log`.
- GIN-distilled K=5 ensemble: `results/distill_ensemble_ames_1778913284.json`, log `results/distill_run.log`.
- Direct-label ensemble recourse: `results/eval_recourse_ensemble_1778898137.json`, log `results/eval_recourse_ensemble_1778897125`
- Distilled ensemble recourse: `results/eval_recourse_distilled_1778914339.json`, log `results/eval_recourse_distilled_1778913638.`
- Kazius coverage, direct seed 42: `results/kazius_coverage_seed42_1778897763.json`.
- Kazius coverage, distilled seed 43: `results/kazius_coverage_distilled_seed43_1778913666.json`.
- Case studies: `results/case_studies_1778918915.json`.

A.5 Figure regeneration

All figures are emitted in PDF and PNG by `scripts/make_figures.py`, which reads the latest matching JSON under `results/` and writes to `paper/figures/`. Re-running the script after a new training run picks up the new JSON automatically.